UNIBALSAS – CENTRO UNIVERSITÁRIO DE BALSAS

CURSO DE SISTEMAS DE INFORMAÇÃO

RAFAEL R PONTARA

MATHEUS DE BRITO ARAUJO

PEDRO PAULO T. ARAUJO

PLATAFORMA INTELIGENTE PARA GESTÃO DE ORDENS DE SERVIÇO E DIAGNÓSTICO DE EQUIPAMENTOS ELETRÔNICOS

Documento Técnico do Projeto — Versão 3.0

BALSAS – MA

2026

RAFAEL R PONTARA

MATHEUS DE BRITO ARAUJO

PEDRO PAULO T. ARAUJO

PLATAFORMA INTELIGENTE PARA GESTÃO DE ORDENS DE SERVIÇO E DIAGNÓSTICO DE EQUIPAMENTOS ELETRÔNICOS

Documento Técnico do Projeto — Versão 3.0

Documento técnico apresentado à disciplina de Programação Web do curso de Sistemas de Informação do Centro Universitário de Balsas – UNIBALSAS, como requisito parcial de avaliação, referente à Versão 3.0 do projeto desenvolvido para a empresa Inforcell.

Professor: Marcos David

BALSAS – MA

2026

HISTÓRICO DE VERSÕES

Quadro 1 — Controle de versões do documento

Versão	Data	Descrição	Responsáveis
1.0	ago. 2026; revisão em 03/11/2026	Primeira versão do Documento Técnico: definição do problema, requisitos, personas, histórias de usuário, arquitetura, tecnologias, cronograma e protótipos (Seções 1 a 24). A revisão de 03/11/2026 reorganizou essas seções, com critérios mensuráveis nos requisitos, e detalhou as personas, a arquitetura e as tecnologias.	Equipe do projeto
2.0	05/10/2026	Desenvolvimento da interface (Seção 25) e considerações sobre a versão (Seção 26): estrutura da aplicação, documentação da interface, decisões arquiteturais, tecnologias, testes e correções, evidências, execução e cronograma atualizado. Definição do back-end e da API de IA. Atualização da equipe: saída de Pedro Junior Ubirajara.	Equipe (3 integrantes)
3.0	03/11/2026	Implementação do back-end: arquitetura implementada (Seção 27), alterações em relação ao planejamento (28), modelagem do banco de dados (29), funcionalidades (30), autenticação, autorização e erros (31), integração e endpoints (32), evidências (33), resultados dos testes (34), dificuldades e soluções (35), execução e repositório (36), cronograma (37) e considerações (38). Preenchimento do Quadro 5. Conteúdo das versões anteriores preservado; Seções 25 e 26 com numeração de quadros, figuras e RNF ajustada.	Equipe (3 integrantes)

Fonte: Elaborado pelos autores (2026).

Este documento é atualizado e ampliado a cada TED. Cada nova versão preserva o conteúdo das anteriores, incorpora as novas seções e é registrada neste quadro. As menções à "Versão 1.0" nas Seções 1 a 24 referem-se à etapa de planejamento em que aquele texto foi produzido.

SUMÁRIO

1 IDENTIFICAÇÃO DO PROJETO 4
2 INTRODUÇÃO 4
3 APRESENTAÇÃO DO CLIENTE 5
3.1 Levantamento de necessidades 5
4 PROBLEMA IDENTIFICADO 6
5 JUSTIFICATIVA 6
5.1 Justificativa social 7
6 OBJETIVO GERAL 7
7 OBJETIVOS ESPECÍFICOS 7
8 PROPOSTA DA SOLUÇÃO 8
9 FUNCIONAMENTO DO DIAGNÓSTICO INTELIGENTE 8
10 REQUISITOS FUNCIONAIS 9
11 REQUISITOS NÃO FUNCIONAIS 9
12 PERSONAS 10
12.1 Atendente 10
12.2 Técnico 11
12.3 Administrador 11
13 HISTÓRIAS DE USUÁRIO 11
14 FLUXO DA ORDEM DE SERVIÇO 12
15 ARQUITETURA PROPOSTA 13
16 TECNOLOGIAS 14
17 BASE DE CONHECIMENTO 15
18 POSSÍVEL INTEGRAÇÃO COM HARDWARE 15
19 PROTÓTIPO 16
19.1 Wireframes de baixa fidelidade 16
20 CONTROLE DE VERSÃO 18
21 CRONOGRAMA 18
22 RESULTADOS ESPERADOS 19
23 CONSIDERAÇÕES FINAIS 19
24 ENTREGÁVEIS DO PROJETO 20
25 DESENVOLVIMENTO DA INTERFACE 21
25.1 Estrutura da aplicação 21
25.2 Documentação da interface desenvolvida 22
25.3 Decisões arquiteturais adotadas durante a implementação 25
25.4 Bibliotecas, frameworks e tecnologias efetivamente utilizadas 26
25.5 Testes e correções 26
25.6 Evidências do funcionamento da interface 28
25.7 Execução e versão publicada 32
25.8 Situação atualizada do cronograma 32
26 CONSIDERAÇÕES SOBRE A VERSÃO 2.0 33
27 ARQUITETURA IMPLEMENTADA 34
27.1 Visão geral 34
27.2 Organização do código 35
27.3 Tecnologias efetivamente utilizadas 35

28 ALTERAÇÕES EM RELAÇÃO AO PLANEJAMENTO 36
29 MODELAGEM DO BANCO DE DADOS 37
29.1 Modelo entidade-relacionamento 37
29.2 Dicionário das tabelas 39
29.3 Scripts de criação e configuração 39
30 PRINCIPAIS FUNCIONALIDADES IMPLEMENTADAS 39
31 AUTENTICAÇÃO, AUTORIZAÇÃO E TRATAMENTO DE ERROS 41
31.1 Autenticação 41
31.2 Autorização 41
31.3 Validações e tratamento de erros 41
32 ARQUITETURA DE INTEGRAÇÃO E ENDPOINTS 42
32.1 Integrações 42
32.2 Integração com o serviço de IA 42
32.3 Endpoints disponibilizados 43
32.4 Documentação técnica dos endpoints 45
33 EVIDÊNCIAS DA INTEGRAÇÃO ENTRE OS COMPONENTES 45
34 RESULTADOS DOS TESTES 47
34.1 Ambiente e abordagem 47
34.2 Resultados 47
34.3 Situação dos requisitos não funcionais e limitações dos testes 48
35 PRINCIPAIS DIFICULDADES E SOLUÇÕES 49
36 EXECUÇÃO E REPOSITÓRIO 49
36.1 Instalação e execução 49
36.2 Controle de versão 50
36.3 Entregáveis da Versão 3.0 50
37 SITUAÇÃO ATUALIZADA DO CRONOGRAMA 50
38 CONSIDERAÇÕES SOBRE A VERSÃO 3.0 51
REFERÊNCIAS 52

1 IDENTIFICAÇÃO DO PROJETO

Esta seção identifica a instituição, a equipe, o cliente atendido e os artefatos entregues junto com este documento.

Quadro 2 — Identificação do projeto

Item	Descrição
Instituição	Centro Universitário de Balsas – UNIBALSAS, Balsas/MA
Curso / Disciplina	Sistemas de Informação / Programação Web
Professor	Marcos David
Projeto	Plataforma Inteligente para Gestão de Ordens de Serviço e Diagnóstico de Equipamentos Eletrônicos
Cliente	Inforcell — assistência técnica de celulares e computadores, Balsas/MA
Protótipo (Figma)	https://www.figma.com/design/n6lJH8wOyIVzayCsOiWnGk/Projeto-de-I.A-de-Assistencia-para-equipamentos-eletronicos
Repositório (GitHub)	https://github.com/Rafaelpontara/inforcell-os-diagnostico

Fonte: Elaborado pelos autores (2026).

A equipe e a divisão de responsabilidades estão descritas no Quadro 3.

Quadro 3 — Equipe e responsabilidades

Integrante	Responsabilidade principal
Rafael R Pontara	Inteligência Artificial e base de conhecimento; qualidade, documentação e contato com o cliente
Matheus de Brito Araujo	Back-end (API REST, regras de negócio e banco de dados)
Pedro Paulo T. Araujo	Front-end (interface web e prototipação)

Fonte: Elaborado pelos autores (2026).

2 INTRODUÇÃO

O desenvolvimento de sistemas de informação tem contribuído para a modernização e a organização de diferentes tipos de negócio, incluindo empresas que trabalham com a manutenção de equipamentos eletrônicos.

No setor de assistência técnica, além da necessidade de controlar clientes, equipamentos e Ordens de Serviço (OS), existe uma dificuldade relacionada ao processo de diagnóstico: um mesmo sintoma pode ter diferentes causas, o que obriga o técnico a realizar uma sequência de testes até identificar a origem do problema.

Diante desse cenário, este projeto propõe o desenvolvimento de uma plataforma web integrada a recursos de Inteligência Artificial (IA), destinada ao gerenciamento de Ordens de Serviço e ao auxílio no diagnóstico de equipamentos eletrônicos. O cliente atendido é a Inforcell, assistência técnica localizada em Balsas – MA.

Este documento corresponde à Versão 1.0 do Documento Técnico do Projeto e reúne o planejamento inicial: problema, público-alvo, requisitos, arquitetura, tecnologias, cronograma e protótipos. Seguindo a orientação de Sommerville (2019), os requisitos foram levantados junto ao cliente e organizados em funcionais e não funcionais, servindo de base para as próximas etapas do desenvolvimento.

3 APRESENTAÇÃO DO CLIENTE

A Inforcell é uma empresa do segmento de assistência técnica localizada em Balsas – MA, que realiza principalmente a manutenção de celulares e computadores. Além dos serviços de reparo, a loja comercializa aparelhos, acessórios e películas, e atende seus clientes de forma presencial e por canais digitais, como WhatsApp e Instagram.

Hoje, parte da comunicação e do acompanhamento dos serviços é feita pelo WhatsApp. Embora prático, esse aplicativo não foi criado para gerenciar Ordens de Serviço: informações sobre equipamentos, clientes, defeitos e andamento dos reparos ficam espalhadas em conversas, o que dificulta consultas e o histórico.

Quadro 4 — Perfil do cliente

Item	**Descrição**
Razão social / nome fantasia	Inforcell
Localização	Balsas – Maranhão
Segmento	Assistência técnica de celulares e computadores; venda de aparelhos e acessórios
Canais de atendimento	Presencial, WhatsApp e Instagram
Equipe	[PREENCHER: nº de técnicos e atendentes]
Volume médio de atendimentos	[PREENCHER: aprox. de OS por mês]
Responsável / contato	[PREENCHER: nome e função do responsável]

Fonte: Elaborado pelos autores com base em entrevista com o cliente (2026).

3.1 Levantamento de necessidades

As necessidades foram levantadas por meio de reuniões e entrevistas semiestruturadas com o cliente, complementadas pela observação da rotina de atendimento e reparo da loja. O Quadro 5 resume esses encontros.

Quadro 5 — Reuniões e entrevistas com o cliente

Data	Participantes	Técnica	Principais pontos levantados
27/09	Dona da empresa; Rafael (técnico)	Reunião inicial	Apresentação da proposta; uso do WhatsApp para acompanhar serviços; dificuldade de manter histórico.
27/09	Rafael (técnico)	Entrevista semiestruturada	Processo de diagnóstico; um mesmo sintoma com várias causas; dependência da experiência do técnico.
28/09	Fernanda (atendente)	Observação da rotina	Recebimento do aparelho, registro do defeito, aprovação de orçamento e entrega.

Fonte: Elaborado pelos autores (2026).

4 PROBLEMA IDENTIFICADO

O principal problema identificado é a dificuldade em determinar as possíveis causas das falhas nos equipamentos. Um sintoma não representa necessariamente uma única falha. Um computador que não liga, por exemplo, pode ter problemas na alimentação, na fonte, na memória, na placa-mãe, no processador ou no armazenamento. Da mesma forma, um celular que não liga pode apresentar defeito na bateria, no circuito de carga, na alimentação da placa, em componentes eletrônicos ou no software.

Por isso, o técnico precisa estabelecer uma sequência de testes para eliminar possibilidades até chegar à causa provável, processo que depende muito de sua experiência individual. Somado a isso, o uso do WhatsApp como ferramenta de acompanhamento dificulta a centralização das informações das Ordens de Serviço.

Os principais problemas identificados são:

- dificuldade em identificar as possíveis causas de um mesmo defeito;
- necessidade de realizar diversos testes até chegar ao diagnóstico;
- dependência da experiência individual do técnico;
- dificuldade de encontrar informações técnicas durante o reparo;
- ausência de uma ferramenta específica para apoiar o diagnóstico;
- uso do WhatsApp para comunicação e acompanhamento das OS;
- falta de um histórico estruturado de equipamentos e reparos.

5 JUSTIFICATIVA

A plataforma se justifica pela necessidade de oferecer à Inforcell uma ferramenta que organize as Ordens de Serviço e, ao mesmo tempo, apoie o processo de diagnóstico. Centralizar as informações hoje dispersas em conversas permite consultar rapidamente o histórico de clientes, equipamentos e serviços.

A integração com Inteligência Artificial não tem o objetivo de fornecer respostas genéricas, mas de conduzir o técnico por um processo estruturado. Diante do problema "computador não liga", por exemplo, a plataforma poderá identificar o equipamento, levantar as causas possíveis, indicar o primeiro teste e a medição necessária, apresentar o resultado esperado, registrar e interpretar o resultado obtido e sugerir o próximo procedimento até o diagnóstico. Assim, a ferramenta atua como apoio à decisão técnica, sem substituir o profissional responsável pelo reparo.

5.1 Justificativa social

Além do ganho operacional para a Inforcell, o projeto tem relevância social para o município de Balsas – MA. Ao tornar o diagnóstico mais estruturado e menos dependente da experiência individual, a plataforma pode contribuir para a qualificação da mão de obra local, servindo de apoio ao aprendizado de técnicos em formação.

A organização das Ordens de Serviço também tende a aumentar a transparência do atendimento, já que o cliente passa a ter mais clareza sobre o andamento do reparo de seu equipamento. Por fim, um diagnóstico mais preciso reduz a troca desnecessária de peças e favorece o reparo em vez do descarte precoce, contribuindo, ainda que indiretamente, para a redução do lixo eletrônico gerado na região.

6 OBJETIVO GERAL

Desenvolver uma plataforma web para gerenciamento de Ordens de Serviço integrada a um assistente de Inteligência Artificial capaz de auxiliar técnicos no diagnóstico e na manutenção de equipamentos eletrônicos.

7 OBJETIVOS ESPECÍFICOS

- desenvolver uma plataforma web para gerenciamento de Ordens de Serviço;
- permitir o cadastro e o gerenciamento de clientes e equipamentos;
- permitir o acompanhamento do status das OS;
- disponibilizar uma área específica para os técnicos;
- desenvolver um assistente de IA que identifique possíveis causas de defeitos e apresente procedimentos de diagnóstico passo a passo;
- permitir o registro e a interpretação de medições e resultados de testes;
- armazenar o histórico de diagnósticos e reparos;
- estudar a viabilidade de integração com equipamentos por USB, quando tecnicamente possível.

8 PROPOSTA DA SOLUÇÃO

A solução proposta é uma plataforma web dividida em dois grandes módulos, apresentados no Quadro 6.

Quadro 6 — Módulos da plataforma

Módulo	Responsabilidades
Gestão da assistência técnica	Clientes, equipamentos, Ordens de Serviço, técnicos, status dos serviços, histórico e registros de manutenção.
Assistência inteligente ao técnico	Análise de sintomas, identificação de causas possíveis, diagnóstico guiado passo a passo, orientação de medições, interpretação de resultados e registro das etapas.

Fonte: Elaborado pelos autores (2026).

O fluxo geral de uso começa no atendimento: o atendente cadastra o cliente e o equipamento e abre a OS; o técnico recebe a OS, consulta o assistente de IA, realiza os testes indicados e informa os resultados; a IA interpreta cada resultado e sugere o próximo teste até o diagnóstico; em seguida, o reparo é registrado e a OS é finalizada e entregue ao cliente.

9 FUNCIONAMENTO DO DIAGNÓSTICO INTELIGENTE

O diagnóstico assistido é a principal funcionalidade da plataforma. Ele funciona como um diálogo em etapas, no qual cada resposta do técnico define o próximo passo. O Quadro 7 ilustra um exemplo.

Quadro 7 — Exemplo de diagnóstico guiado (equipamento: computador; sintoma: não liga)

Etapa	Orientação da IA	Resultado informado	Interpretação / próximo passo
1 – Alimentação	Verificar se o equipamento recebe alimentação. Medir a saída da fonte com multímetro.	Dentro do esperado	Alimentação OK. Avançar para o circuito seguinte.
2 – Circuito de entrada	Medir a tensão na linha principal da placa.	Abaixo do esperado	Possível anormalidade nesta etapa. Verificar curto-circuito ou componente danificado na linha.
3 – Componentes	Testar os componentes da linha indicada.	Componente em curto	Registrar diagnóstico e sugerir o reparo.

Fonte: Elaborado pelos autores (2026).

Dessa forma, o diagnóstico é dinâmico e muda conforme os resultados encontrados. Todas as etapas ficam registradas na OS, formando um histórico que poderá ser consultado em reparos futuros.

10 REQUISITOS FUNCIONAIS

Segundo Sommerville (2019), requisitos funcionais descrevem o que o sistema deve fazer, ou seja, os serviços que deve oferecer e como deve reagir a determinadas entradas. O Quadro 8 apresenta os requisitos funcionais levantados, com prioridade definida pela técnica MoSCoW (Essencial, Importante, Desejável).

Quadro 8 — Requisitos funcionais

Código	Requisito	Prioridade
RF01	Permitir o cadastro de usuários.	Essencial
RF02	Possuir níveis de acesso (administrador, atendente, técnico).	Essencial
RF03	Permitir o cadastro de clientes.	Essencial
RF04	Permitir o cadastro de equipamentos vinculados a clientes.	Essencial
RF05	Permitir a criação de Ordens de Serviço.	Essencial
RF06	Permitir a alteração do status das OS.	Essencial
RF07	Permitir atribuir uma OS a um técnico.	Essencial
RF08	Permitir que o técnico visualize as informações da OS.	Essencial
RF09	Permitir que o técnico registre informações do diagnóstico.	Essencial
RF10	Disponibilizar um assistente de Inteligência Artificial.	Essencial
RF11	A IA deverá analisar os sintomas informados.	Essencial
RF12	A IA deverá apresentar possíveis causas para o problema.	Essencial
RF13	A IA deverá fornecer procedimentos de diagnóstico passo a passo.	Essencial
RF14	Permitir o registro das medições realizadas.	Essencial
RF15	Permitir o registro dos resultados dos testes.	Essencial
RF16	A IA deverá usar os resultados informados para orientar o diagnóstico.	Essencial
RF17	Armazenar o histórico dos diagnósticos.	Importante
RF18	Permitir registrar o reparo realizado.	Importante
RF19	Permitir consultar o histórico do equipamento.	Importante
RF20	Gerar comprovante da OS (entrada e entrega) para o cliente.	Importante
RF21	Permitir buscar OS por cliente, número, equipamento ou IMEI/número de série.	Importante
RF22	Prever integrações futuras com equipamentos eletrônicos.	Desejável
RF23	Quando tecnicamente possível, comunicar-se com equipamentos por USB.	Desejável

Fonte: Elaborado pelos autores (2026).

11 REQUISITOS NÃO FUNCIONAIS

Requisitos não funcionais são restrições sobre os serviços oferecidos pelo sistema, como desempenho, segurança e usabilidade (SOMMERVILLE, 2019). Para que possam ser verificados nos testes, foram definidos com critérios mensuráveis, apresentados no Quadro 9.

Quadro 9 — Requisitos não funcionais

Código	Categoria	Requisito
RNF01	Usabilidade	A abertura de uma OS deverá ser concluída em até 2 minutos por um atendente após breve treinamento.
RNF02	Segurança	Os usuários deverão se autenticar com e-mail e senha; as senhas serão armazenadas com hash (bcrypt).
RNF03	Segurança	O acesso às funções deverá respeitar o perfil do usuário, com sessão via token JWT que expira em até 8 horas.
RNF04	Privacidade	Os dados pessoais dos clientes deverão ser tratados conforme a LGPD (Lei nº 13.709/2018) e não serão enviados ao serviço de IA.
RNF05	Desempenho	As telas de cadastro e listagem deverão carregar em até 3 segundos em conexão de 10 Mbps.
RNF06	Desempenho	A primeira resposta do assistente de IA deverá ser exibida em até 15 segundos.
RNF07	Compatibilidade	A plataforma deverá funcionar nas duas últimas versões de Chrome, Edge e Firefox, inclusive em telas a partir de 360 px de largura.
RNF08	Segurança	Toda a comunicação deverá ocorrer por HTTPS.
RNF09	Manutenibilidade	O código deverá ser versionado no GitHub e seguir padrão de organização em camadas (rotas, controladores, serviços).
RNF10	Disponibilidade	Deverá ser realizado backup diário do banco de dados.

Fonte: Elaborado pelos autores (2026).

12 PERSONAS

Personas são representações fictícias, mas fundamentadas em dados reais, dos usuários de um sistema. Cooper (1999) propõe seu uso para que a equipe projete a interface pensando em objetivos concretos de pessoas específicas, e não em um usuário genérico. As personas abaixo foram construídas a partir das entrevistas com a Inforcell.

12.1 Atendente

Quadro 10 — Persona atendente

Persona	Juliana, 24 anos — atendente
Perfil	Trabalha no balcão no turno da tarde. Usa o celular com facilidade, mas não tem formação técnica.
Objetivos	Registrar rapidamente o aparelho e o defeito relatado; informar ao cliente o andamento do serviço sem precisar interromper o técnico.
Frustrações	Procurar informações em conversas antigas do WhatsApp; não saber em que etapa está um reparo quando o cliente liga.
Como usará a plataforma	Cadastrar clientes e equipamentos, abrir OS, consultar status e gerar comprovantes.

Fonte: Elaborado pelos autores (2026).

12.2 Técnico

Quadro 11 — Persona técnico

Persona	Carlos, 31 anos — técnico em eletrônica
Perfil	Oito anos de experiência com celulares, domina a troca de peças, mas tem menos prática com reparo em placa de notebook.
Objetivos	Chegar ao diagnóstico com menos testes; consultar como foram resolvidos casos parecidos.
Frustrações	Um mesmo sintoma ter várias causas; perder tempo procurando esquemas e valores de referência em fóruns.
Como usará a plataforma	Visualizar suas OS, usar o assistente de IA, registrar medições, resultados e o reparo realizado.

Fonte: Elaborado pelos autores (2026).

12.3 Administrador

Quadro 12 — Persona administrador

Persona	Roberto, 40 anos — proprietário
Perfil	Gerencia a loja, atende clientes e também realiza reparos quando há demanda.
Objetivos	Saber quantos equipamentos estão em cada etapa; acompanhar a produtividade e o histórico dos serviços.
Frustrações	Falta de números confiáveis sobre os atendimentos; informações espalhadas em vários aparelhos.
Como usará a plataforma	Gerenciar usuários, acompanhar o painel de OS e consultar históricos.

Fonte: Elaborado pelos autores (2026).

13 HISTÓRIAS DE USUÁRIO

As histórias de usuário descrevem funcionalidades sob o ponto de vista de quem as utiliza, no formato "Como [papel], quero [ação], para [benefício]". Cohn (2004) recomenda que cada história tenha critérios de aceitação, que definem quando ela pode ser considerada concluída. O Quadro 13 relaciona cada história aos requisitos funcionais correspondentes.

Quadro 13 — Histórias de usuário

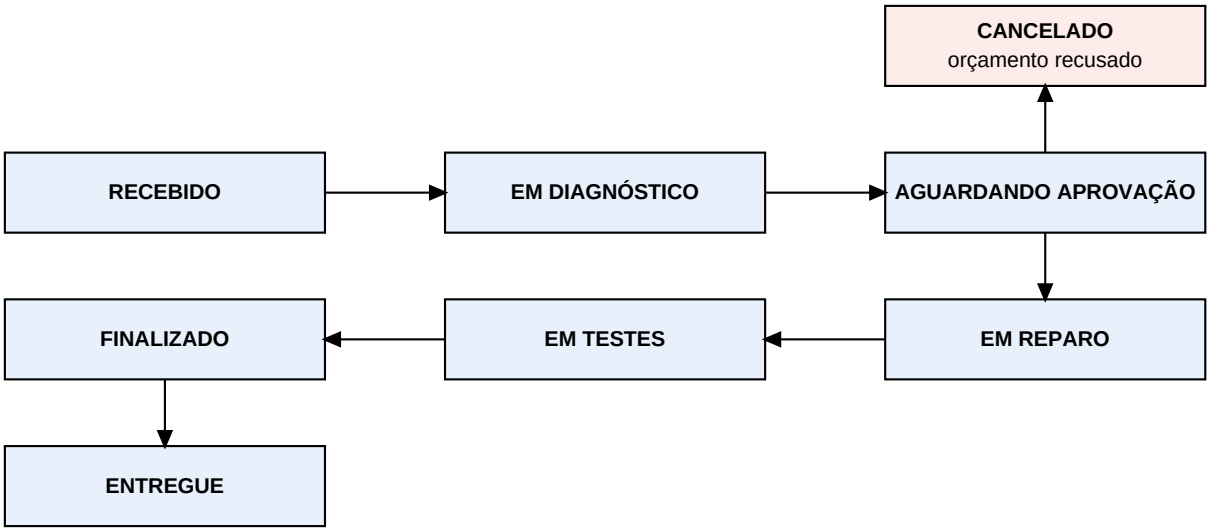
História	Descrição	Critérios de aceitação	RF
HU01 — Acessar o sistema	Como usuário, quero entrar com e-mail e senha, para acessar somente as funções do meu perfil.	Dado que informei credenciais válidas, quando clico em Entrar, então sou levado ao painel do meu perfil; com credenciais inválidas, uma mensagem de erro é exibida.	RF01, RF02
HU02 — Cadastrar cliente e equipamento	Como atendente, quero cadastrar o cliente e seu equipamento, para não precisar digitar os dados novamente em atendimentos futuros.	Dado um CPF ou telefone já cadastrado, quando o informo, então os dados do cliente são carregados; um cliente pode ter vários equipamentos.	RF03, RF04
HU03 — Criar Ordem de Serviço	Como atendente, quero criar uma OS para registrar o equipamento recebido, para que o técnico possa iniciar o diagnóstico.	Dado cliente e equipamento cadastrados, quando registro o defeito relatado, então a OS recebe número sequencial, status RECEBIDO e pode gerar comprovante.	RF05, RF20
HU04 — Visualizar Ordem de Serviço	Como técnico, quero visualizar as OS atribuídas a mim, para compreender o problema relatado pelo cliente.	Dado que estou autenticado como técnico, quando abro minha área, então vejo apenas as OS atribuídas a mim, com defeito relatado e histórico do equipamento.	RF07, RF08, RF19
HU05 — Solicitar auxílio da IA	Como técnico, quero informar o problema do equipamento para a IA, para receber causas possíveis e orientações de diagnóstico.	Dado uma OS aberta, quando inicio o assistente, então ele apresenta ao menos três causas possíveis ordenadas por probabilidade e o primeiro teste a realizar.	RF10–RF13
HU06 — Registrar medição e seguir o diagnóstico	Como técnico, quero registrar o resultado de cada teste, para que a IA indique o próximo passo.	Dado um teste em andamento, quando informo o valor medido, então ele fica salvo na OS e a IA apresenta a interpretação e o próximo teste.	RF14–RF16
HU07 — Registrar reparo	Como técnico, quero registrar o diagnóstico final e o reparo realizado, para manter o histórico do equipamento.	Dado o diagnóstico concluído, quando registro peças e serviços, então a OS passa para EM TESTES e o registro aparece no histórico do equipamento.	RF09, RF17, RF18
HU08 — Acompanhar Ordens de Serviço	Como administrador, quero ver um painel com as OS por status, para saber quais equipamentos estão em manutenção.	Dado que estou autenticado como administrador, quando abro o painel, então vejo a quantidade de OS em cada status e posso filtrar por técnico e período.	RF06, RF21

Fonte: Elaborado pelos autores (2026).

14 FLUXO DA ORDEM DE SERVIÇO

A Figura 1 apresenta os status pelos quais uma OS passa, do recebimento à entrega. Caso o cliente recuse o orçamento na etapa AGUARDANDO APROVAÇÃO, a OS é encerrada com o status CANCELADO e o equipamento é devolvido. O fluxo poderá ser ajustado conforme as necessidades reais da Inforcell.

Figura 1 — Fluxo de status da Ordem de Serviço

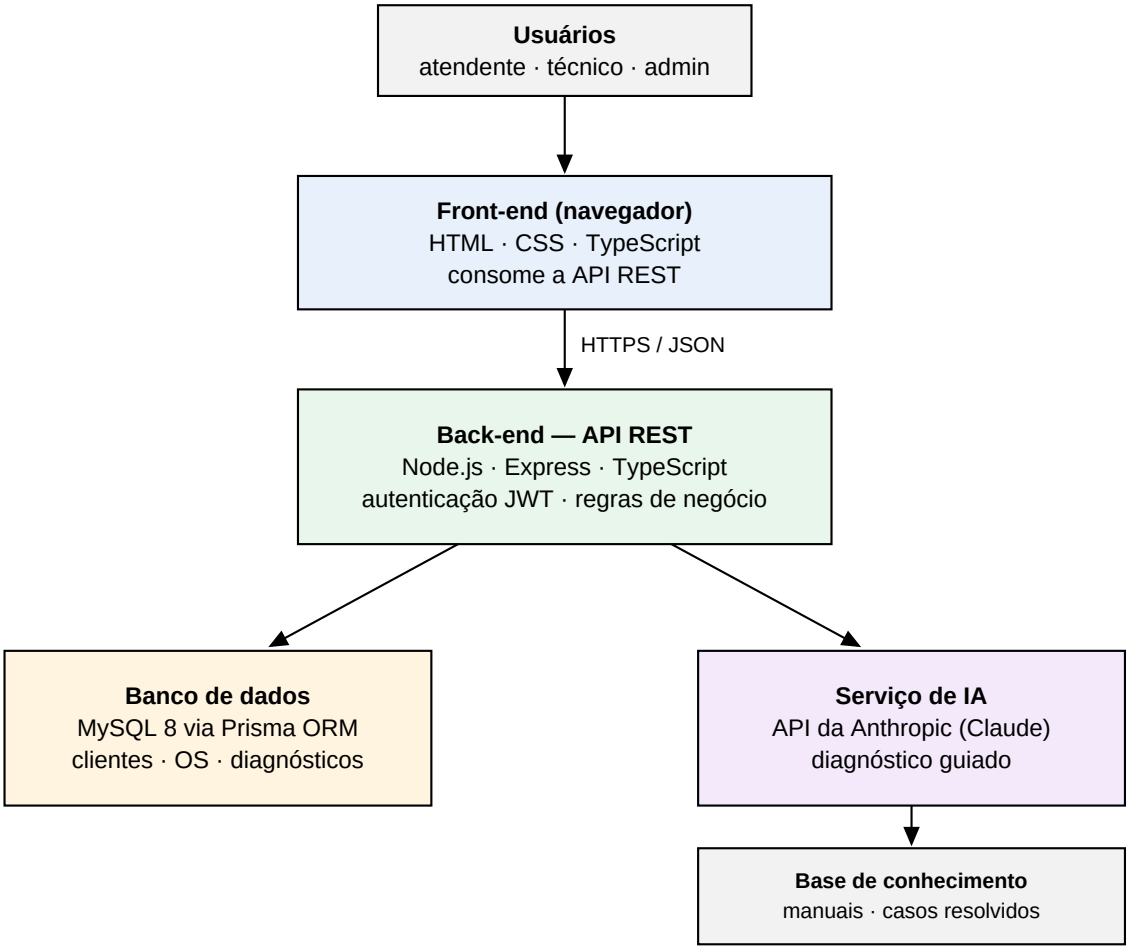


Fonte: Elaborado pelos autores (2026).

15 ARQUITETURA PROPOSTA

A plataforma seguirá uma arquitetura cliente-servidor em camadas, como mostra a Figura 2. O front-end, executado no navegador, se comunica por HTTPS com uma API REST no back-end. A API concentra as regras de negócio e o controle de acesso, persiste os dados em MySQL e intermedia toda a comunicação com o serviço de IA.

Figura 2 — Arquitetura proposta da plataforma



Fonte: Elaborado pelos autores (2026).

Duas decisões de arquitetura merecem destaque. Primeiro, o front-end nunca acessa a API de IA diretamente: a chave de acesso fica apenas no servidor, o que evita sua exposição. Segundo, antes de cada consulta, o back-end envia à IA apenas os dados técnicos do equipamento (modelo, sintoma e medições), sem nome, telefone ou outros dados pessoais do cliente, em atendimento ao RNF04. A base de conhecimento técnico é consultada pelo back-end para complementar o contexto enviado à IA.

16 TECNOLOGIAS

O Quadro 14 apresenta as tecnologias definidas para cada camada e a justificativa de cada escolha.

Quadro 14 — Tecnologias por camada

Camada	Tecnologia	Justificativa
Front-end	HTML5, CSS3, TypeScript	Padrões da web com suporte em todos os navegadores modernos (MDN WEB DOCS, 2026); a tipagem do TypeScript reduz erros e facilita a manutenção (MICROSOFT, 2026).
Back-end	Node.js com Express e TypeScript	Mesma linguagem do front-end, o que facilita o trabalho da equipe; ecossistema amplo para criação de APIs REST; tecnologia trabalhada na disciplina.
Acesso a dados	Prisma ORM	Mapeamento tipado das tabelas, migrações versionadas e proteção contra injeção de SQL.
Banco de dados	MySQL 8	Banco relacional adequado aos dados estruturados de clientes, equipamentos e OS (ORACLE CORPORATION, 2026).
Inteligência Artificial	API da Anthropic (modelos Claude)	Capacidade de raciocínio em etapas, análise de imagens (fotos de placas) e de documentação técnica longa. Escolhida após testes com a Azure, que não atendeu por limitação de recursos e erro na estimativa de uso.
Autenticação	JWT e bcrypt	Sessões sem estado no servidor e armazenamento seguro de senhas.
Prototipação	Figma	Ferramenta colaborativa para protótipos de alta fidelidade (FIGMA, 2026).
Versionamento	Git e GitHub	Controle de versões e trabalho colaborativo (GITHUB, 2026).

Fonte: Elaborado pelos autores (2026).

17 BASE DE CONHECIMENTO

Uma evolução importante da plataforma será a criação de uma base de conhecimento técnico, que poderá conter manuais, documentação de equipamentos, procedimentos de medição, especificações de componentes e registros de problemas conhecidos e suas soluções. A IA utilizará essas informações para produzir respostas mais contextualizadas. Os diagnósticos concluídos na própria plataforma também alimentarão essa base, tornando-a mais útil à medida que a Inforcell a utiliza.

18 POSSÍVEL INTEGRAÇÃO COM HARDWARE

Como funcionalidade futura, a plataforma poderá se integrar a equipamentos por USB ou outras interfaces compatíveis, permitindo a coleta automática de informações quando houver suporte técnico para isso (Figura 3). Uma possibilidade em estudo é a Web Serial API, disponível em navegadores baseados em Chromium, que permite a uma página web comunicar-se com dispositivos seriais, como fontes de bancada e multímetros com saída USB.

Figura 3 — Fluxo conceitual de integração via USB (funcionalidade futura)



Fonte: Elaborado pelos autores (2026).

Essa funcionalidade depende das características e dos protocolos de comunicação de cada equipamento e, por isso, será tratada como dependente de compatibilidade técnica e desenvolvida de forma progressiva.

19 PROTÓTIPO

A equipe desenvolveu um protótipo de alta fidelidade no Figma, disponível em: https://www.figma.com/design/n6lJH8wOyIVzayCsOiWnGk/Projeto-de-I.A-de-Assistencia-para-equipamentos-eletronicos.

As telas previstas são: login, dashboard, lista de Ordens de Serviço, cadastro de cliente, cadastro de equipamento, criação de OS, detalhes da OS, área do técnico, assistente de IA, diagnóstico, registro de medições e histórico do equipamento.

19.1 Wireframes de baixa fidelidade

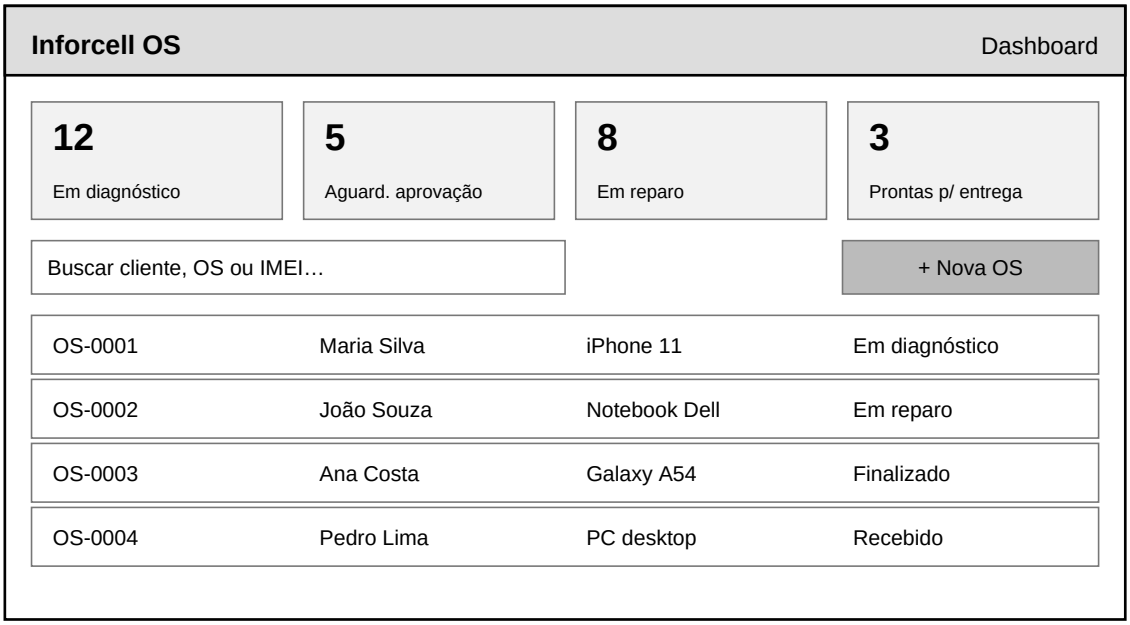
As figuras a seguir apresentam os wireframes das principais telas, que representam a estrutura geral da interface. As versões visuais de alta fidelidade estão no protótipo do Figma.

Figura 4 — Wireframe da tela de login



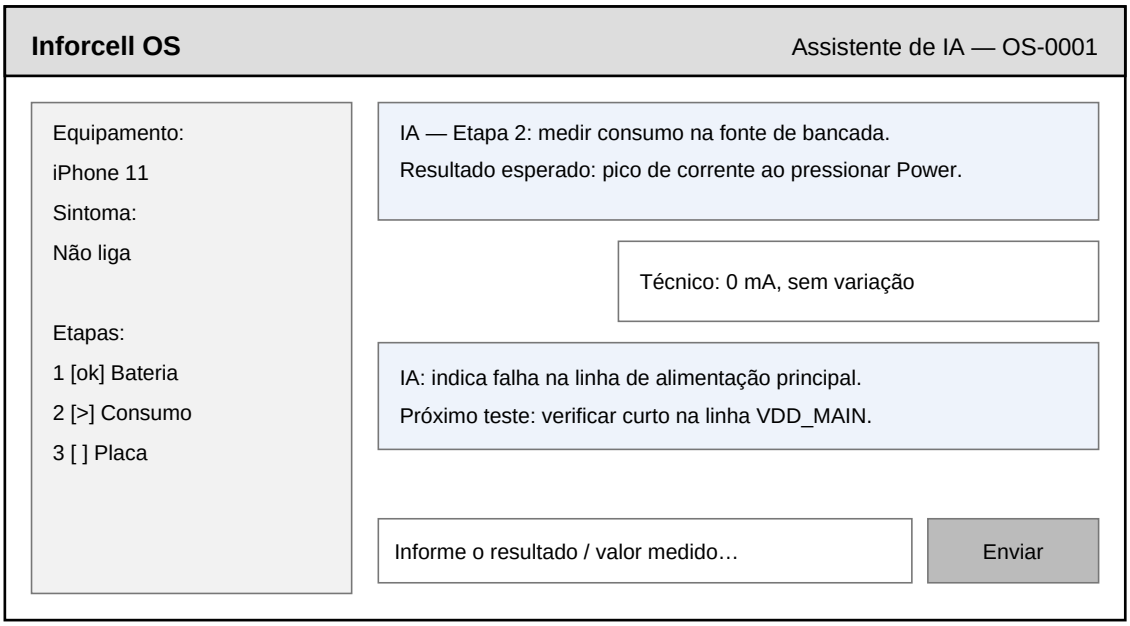
Fonte: Elaborado pelos autores (2026).

Figura 5 — Wireframe do dashboard e lista de OS



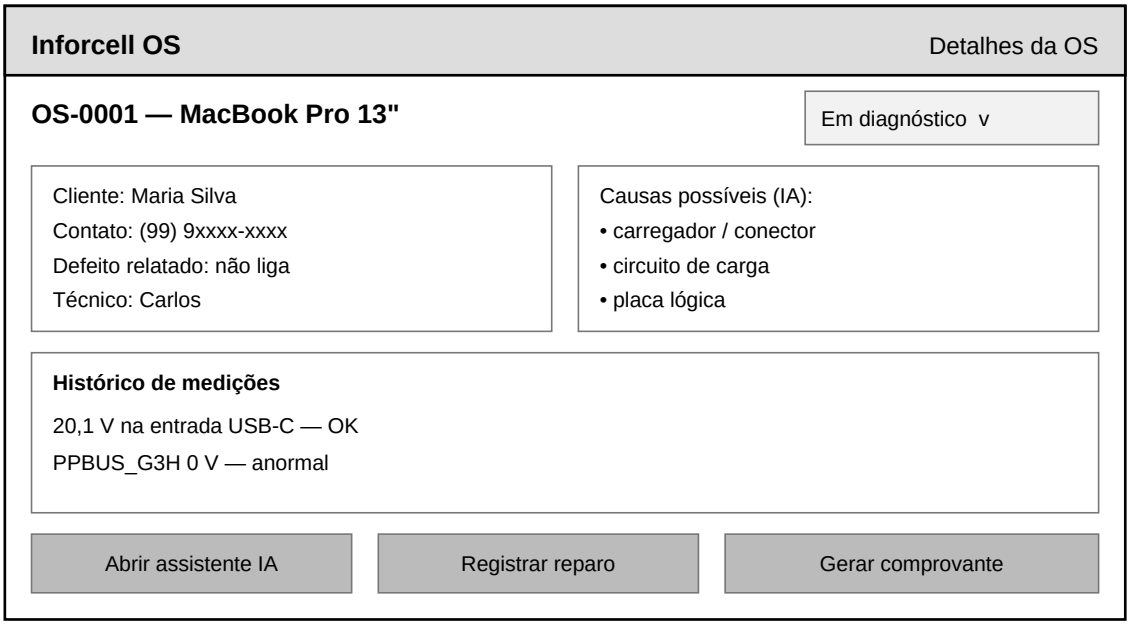
Fonte: Elaborado pelos autores (2026).

Figura 6 — Wireframe do assistente de IA (diagnóstico guiado)



Fonte: Elaborado pelos autores (2026).

Figura 7 — Wireframe dos detalhes da OS



Fonte: Elaborado pelos autores (2026).

20 CONTROLE DE VERSÃO

O código-fonte é mantido no GitHub, no repositório https://github.com/Rafaelpontara/inforcell-os-diagnostico. A equipe adotará o seguinte fluxo de trabalho:

- a branch *main* contém apenas versões estáveis, entregues ao final de cada etapa;
- a branch *develop* integra as funcionalidades em desenvolvimento;
- cada funcionalidade é desenvolvida em uma branch própria (por exemplo, *feature/cadastro-cliente*) e integrada por *pull request* revisado por outro integrante;
- as mensagens de commit seguem o padrão Conventional Commits (por exemplo, *feat:*, *fix:*, *docs:*).

21 CRONOGRAMA

O Quadro 15 apresenta o cronograma simplificado do projeto ao longo do semestre letivo de 2026, organizado de forma incremental, como recomenda Pressman (2016), de modo que cada etapa entregue uma parte funcional do sistema.

Quadro 15 — Cronograma simplificado

Etapa	Atividade	Período	Situação
1	Identificação do problema	Ago/2026	Concluída
2	Levantamento com o cliente	Ago/2026	Concluída
3	Definição da proposta	Ago/2026	Concluída
4	Levantamento de requisitos	Ago–Set/2026	Concluída
5	Prototipação (Figma)	Set–Out/2026	Concluída
6	Definição da arquitetura e do back-end	Out/2026	Concluída
7	Definição da API de IA	Out/2026	Concluída
8	Desenvolvimento da interface	Out–Nov/2026	Em andamento
9	Desenvolvimento do back-end e banco de dados	Nov/2026	Em andamento
10	Gerenciamento de OS	Nov/2026	Pendente
11	Integração com a IA e diagnóstico guiado	Nov–Dez/2026	Pendente
12	Testes e correções	Dez/2026	Pendente
13	Documentação final	Dez/2026	Pendente
14	Apresentação ao cliente e à banca	Dez/2026	Pendente

Fonte: Elaborado pelos autores (2026).

22 RESULTADOS ESPERADOS

Espera-se que a plataforma melhore a organização dos serviços da Inforcell e ofereça aos técnicos uma ferramenta adicional de apoio ao diagnóstico. Entre os resultados esperados estão:

- centralização das informações das Ordens de Serviço, hoje dispersas no WhatsApp;
- maior facilidade no acompanhamento dos serviços pelo atendimento e pela gerência;
- histórico estruturado de equipamentos, diagnósticos e reparos;
- diagnósticos mais rápidos e organizados, com menos testes desnecessários;
- redução da dependência da experiência individual do técnico;
- base para evoluções futuras, como diagnóstico automatizado e integração com equipamentos.

23 CONSIDERAÇÕES FINAIS

Este documento apresentou o planejamento inicial de uma plataforma web voltada a uma necessidade real identificada na Inforcell, assistência técnica de Balsas – MA. A solução une duas demandas relacionadas: a gestão das Ordens de Serviço e o apoio inteligente ao diagnóstico de equipamentos eletrônicos.

Com a arquitetura e as tecnologias definidas, as próximas etapas concentram-se no desenvolvimento da interface, da API e da integração com a IA. Esta Versão 1.0 será atualizada nas TEDs seguintes, preservando o conteúdo já produzido e incorporando as novas seções desenvolvidas ao longo do semestre.

24 ENTREGÁVEIS DO PROJETO

Além deste documento, fazem parte da entrega:

Quadro 16 — Entregáveis

Entregável	Link
Protótipo (Figma)	https://www.figma.com/design/n6lJH8wOyIVzayCsOiWnGk/Projeto-de-I.A-de-Assistencia-para-equipamentos-eletronicos
Repositório (GitHub)	https://github.com/Rafaelpontara/inforcell-os-diagnostico

Fonte: Elaborado pelos autores (2026).

25 DESENVOLVIMENTO DA INTERFACE

Nota da Versão 3.0: as Seções 25 e 26 são preservadas da Versão 2.0 e descrevem a situação em 05/10/2026. Foram ajustadas apenas a numeração de quadros e figuras e os códigos de RNF, para acompanhar a revisão das Seções 1 a 24.

Esta seção, acrescentada na Versão 2.0, documenta a implementação da interface da aplicação, conforme o planejamento definido na 1ª versão deste documento. O desenvolvimento começou pela página inicial (página de apresentação), que apresenta a plataforma e conduz o usuário às demais telas previstas na Seção 19. A implementação seguiu o protótipo de alta fidelidade do Figma e as tecnologias de front-end da Seção 16 (HTML, CSS, JavaScript e TypeScript), priorizando a experiência do usuário, a organização do código, a responsividade e a acessibilidade.

Nesta etapa, a interface ainda não se comunica com o backend nem com a API de Inteligência Artificial. As telas de login, dashboard, Ordens de Serviço e diagnóstico (Figuras 4 a 7) serão implementadas nas próximas etapas, reaproveitando a estrutura, os estilos e os componentes descritos a seguir.

25.1 Estrutura da aplicação

O código da interface fica na pasta frontend/ do repositório GitHub informado na Seção 1. Os arquivos-fonte (HTML, CSS e TypeScript) ficam separados do JavaScript gerado pelo compilador, gravado na pasta dist/. A estrutura implementada é a seguinte:

```
inforcell-os-diagnostico/
├── .github/workflows/pages.yml  publicação automática no GitHub Pages
├── docs/capturas/               evidências de funcionamento da interface
└── frontend/
    ├── index.html        página inicial (estrutura semântica)
    ├── css/styles.css    variáveis de design, componentes e responsividade
    ├── src/main.ts       comportamentos da interface (TypeScript)
    ├── dist/main.js      JavaScript gerado pelo compilador (npm run build)
    ├── assets/           ícone da aplicação
    ├── package.json      scripts de compilação e servidor local
    ├── tsconfig.json     configuração do TypeScript (modo strict)
    ├── README.md         instalação, execução e utilização
    └── CHANGELOG.md      histórico de versões do front-end
```

Quadro 17 — Arquivos do front-end e suas responsabilidades

Arquivo	Responsabilidade
index.html	Estrutura e conteúdo da página inicial com elementos semânticos (header, nav, main, section, article, footer), atributos de acessibilidade e metadados (título, descrição, idioma pt-BR).
css/styles.css	Organizado em sete blocos: variáveis de design (tokens), base, layout, componentes reutilizáveis, seções, animações e responsividade.
src/main.ts	Funções initHeader, initMobileNav, initScrollSpy, initReveal, initCounters, initTerminal e initYear, executadas quando a página termina de carregar.
dist/main.js	Saída do compilador TypeScript, carregada pela página com o atributo defer. Não é editado manualmente.
package.json	Scripts build (compila), watch (recompila a cada alteração), serve (servidor local na porta 5173) e dev (compila e serve).
tsconfig.json	Compilação de src/ para dist/ com destino ES2020, modo strict e verificação de variáveis e parâmetros não utilizados.
README.md e CHANGELOG.md	Instruções de instalação, execução e utilização; registro das alterações por versão.

Fonte: Elaborado pelos autores (2026).

25.2 Documentação da interface desenvolvida

A página inicial adota o nome FIXBOT.AI, utilizado no protótipo de alta fidelidade, e segue sua identidade visual: tema escuro, cor de destaque verde-água, fonte monoespaçada nos títulos (JetBrains Mono) e fonte sem serifa no texto corrido (Inter), em referência ao ambiente de bancada do técnico. O Quadro 18 descreve as seções implementadas.

Quadro 18 — Seções da página inicial

Seção	Conteúdo	Objetivo para o usuário
Cabeçalho	Logotipo, menu (Plataforma, Como funciona, Recursos, Planos, FAQ), indicador do status da IA e botão Entrar. Fixo no topo; no celular, vira menu recolhível.	Orientar a navegação e indicar a seção visível.
Abertura (hero)	Proposta “Do sintoma à causa com um roteiro de testes guiado por IA”, botões de ação e janela ilustrativa de um diagnóstico com causas prováveis, próxima etapa e alerta de segurança.	Explicar rapidamente o que a plataforma faz (Seção 8).
Indicadores	Metas do projeto: 3 sintomas-piloto, meta de acerto, número máximo de etapas por diagnóstico e registro das etapas na OS.	Apresentar o escopo do produto.
Plataforma	Quatro cartões: Ordens de Serviço, Diagnóstico guiado, Histórico/casos semelhantes e Base de conhecimento.	Apresentar os dois grupos de funcionalidades (Seção 8) e a base de conhecimento (Seção 17).
Como funciona	Fluxo em quatro passos: abrir a OS, receber as hipóteses, seguir o roteiro de testes, confirmar e registrar.	Resumir o diagnóstico inteligente (Seção 9).
Recursos	Seis diferenciais (a IA propõe e o técnico decide, alertas de segurança, respostas estruturadas, perfis de acesso e privacidade, responsividade, “não sei” quando falta informação) e um terminal animado que simula um diagnóstico.	Gerar confiança e demonstrar o funcionamento esperado.
Validação	Estratégias de validação com o público-alvo (entrevista, questionário, teste de conceito e usabilidade).	Mostrar que a solução é construída com técnicos.
Planos	Piloto (gratuito), Assistência (valor em pesquisa) e Rede (sob consulta).	Apresentar as opções de uso.
FAQ	Seis perguntas frequentes em blocos expansíveis.	Responder às principais dúvidas.
Chamada final e rodapé	Link para o repositório, navegação, equipe e créditos acadêmicos.	Encaminhar o visitante e identificar o projeto.

Fonte: Elaborado pelos autores (2026).

Os elementos visuais foram construídos como componentes reutilizáveis — botões, cartões, etiquetas, listas com marcação, janela estilo console e blocos expansíveis — que serão aproveitados nas próximas telas. Cores, fontes e espaçamentos foram definidos uma única vez como variáveis CSS (Quadro 19), o que mantém a consistência visual e facilita a manutenção (RNF09).

Quadro 19 — Principais variáveis de design (tokens)

Variável	Valor	Uso
--bg / --bg-alt	#070b10 / #0b1118	Fundo da página e das seções alternadas.
--surface	#0f1720	Fundo de cartões, janelas e blocos.
--text / --muted	#e6edf3 / #8b9aab	Texto principal e texto secundário.
--accent	#19e6c1	Destaque: botões, links ativos, ícones e números.
--warn	#f5a524	Alertas de segurança e etiquetas de etapa.
--font-sans / --font-mono	Inter / JetBrains Mono	Texto corrido / títulos, botões e console.

Fonte: Elaborado pelos autores (2026).

Em relação à usabilidade e acessibilidade (RNF01), seguindo as diretrizes WCAG (W3C, 2023), a interface adota:

- HTML semântico, idioma declarado (pt-BR) e hierarquia de títulos coerente, favorecendo leitores de tela;
- link “Pular para o conteúdo”, primeiro item alcançado pela tecla Tab;
- contorno de foco visível em todos os elementos interativos e áreas de toque com no mínimo 44 px de altura;
- menu do celular com atributos aria-expanded e aria-label atualizados, fechamento pela tecla Esc e retorno do foco ao botão;
- perguntas frequentes com o elemento nativo details, operável por teclado mesmo sem JavaScript;
- respeito à preferência do sistema por movimento reduzido (prefers-reduced-motion), que desativa as animações.

Quanto à responsividade, a interface usa CSS Grid e Flexbox com três pontos de quebra: até 1024 px (grades em duas colunas), até 900 px (menu recolhível e conteúdo em uma coluna) e até 560 px (todas as grades em uma coluna e botões com largura total). Tamanhos de fonte e espaçamentos usam a função clamp(), ajustando-se de forma contínua à largura da tela, o que permite o uso no computador do balcão e no celular da bancada.

Os comportamentos interativos, implementados em TypeScript, são: borda no cabeçalho ao rolar; abertura e fechamento do menu do celular; destaque automático, no menu, da seção visível (com IntersectionObserver); entrada suave dos blocos na tela; contagem animada dos indicadores; e um terminal de demonstração que digita, linha a linha, uma sessão de diagnóstico semelhante ao exemplo da Seção 9, podendo ser reiniciado pelo botão “repetir”.

25.3 Decisões arquiteturais adotadas durante a implementação

Na 1ª versão entregue, o backend e a API de IA estavam “a definir”. Essas definições foram tomadas pela equipe ao longo do projeto e são registradas no Quadro 20, junto com as decisões da implementação da interface (Quadro 21).

Quadro 20 — Atualização da arquitetura inicial (Seção 15)

Camada	**1ª versão**	**Versão 2.0**
Interface web	HTML, CSS, JavaScript e TypeScript	Mantido. Implementada a página inicial (esta seção).
Backend	A definir	Node.js com Express e TypeScript, para que toda a equipe use a mesma linguagem no front-end e no backend.
Banco de dados	MySQL	Mantido.
API de IA	A definir	API da Anthropic (modelo de linguagem), com recuperação de procedimentos da base de conhecimento (RAG). A chamada à IA ficará apenas no backend, protegendo as chaves de acesso.
Publicação	Não prevista	GitHub Pages para a interface, com publicação automática pelo GitHub Actions.

Fonte: Elaborado pelos autores (2026).

Quadro 21 — Decisões da implementação da interface

Decisão	**Justificativa**
HTML, CSS e TypeScript sem framework de interface	Aderência às tecnologias da Seção 16; a página inicial é estática e não exige framework, o que reduz dependências e a curva de aprendizado da equipe.
TypeScript em modo strict, compilado para dist/	Tipagem verificada antes da execução, evitando erros comuns; o JavaScript gerado fica separado do código-fonte.
CSS em blocos, com variáveis e nomes no padrão BEM	Facilita a manutenção e a reutilização nas próximas telas (RNF09); o padrão Bloco__Elemento--Modificador evita conflitos de nomes.
Layout responsivo com três pontos de quebra	Uso no balcão e na bancada; funcionamento em navegadores modernos (RNF07).
Aprimoramento progressivo	O conteúdo funciona sem JavaScript; as animações só são ativadas quando o script carrega (classe js).
Conteúdo baseado nos dados reais do projeto	Números e depoimentos ilustrativos do protótipo foram substituídos por metas e informações reais do projeto, evitando dados fictícios.
Diagnóstico simulado, sem chamada à IA	O terminal usa um roteiro fixo. A integração real será feita pelo backend, conforme o Quadro 20.
Publicação pelo GitHub Pages	Gera um link público de demonstração a cada atualização da branch main, sem custo.

Fonte: Elaborado pelos autores (2026).

25.4 Bibliotecas, frameworks e tecnologias efetivamente utilizadas

Não foi utilizada nenhuma biblioteca JavaScript em tempo de execução: todo o comportamento da página foi escrito pela equipe em TypeScript. O Quadro 22 lista as tecnologias empregadas.

Quadro 22 — Tecnologias utilizadas na interface

Tecnologia	Versão	Uso no projeto
HTML5	—	Estrutura semântica da página.
CSS3	—	Estilos, variáveis, Grid, Flexbox, clamp(), media queries e animações.
TypeScript	5.6 ou superior	Comportamentos da interface, compilados com o tsc.
JavaScript (ES2020)	—	Código gerado pelo compilador e executado pelo navegador.
Node.js e npm	18 ou superior	Instalação do TypeScript e execução dos scripts.
http-server (via npx)	Atual	Servidor local para testes (npm run serve).
Google Fonts	—	Fontes Inter e JetBrains Mono.
Git, GitHub e GitHub Actions	—	Versionamento, pull request e publicação automática (GitHub Pages).
Figma	—	Referência visual (protótipo de alta fidelidade).
Playwright com Chromium	—	Verificação automatizada da interface e geração das capturas de tela.

Fonte: Elaborado pelos autores (2026).

25.5 Testes e correções

A interface foi verificada com um roteiro automatizado no Playwright, executado no navegador Chromium, e por testes manuais da equipe. O Quadro 23 apresenta os testes e os requisitos não funcionais relacionados.

Quadro 23 — Testes realizados na página inicial

Teste	Critério	RNF	Resultado
Compilação	TypeScript compila em modo strict sem erros.	RNF09	Aprovado
Responsividade	Sem rolagem horizontal em 1440, 1024, 820, 390 e 320 px.	RNF07, RNF01	Aprovado
Erros de execução	Nenhum erro de JavaScript em todas as larguras.	RNF05	Aprovado
Navegação por teclado	Primeiro Tab alcança “Pular para o conteúdo”; FAQ abre com Enter.	RNF01	Aprovado
Menu do celular	Abre pelo botão; fecha com Esc devolvendo o foco; ao clicar em um link, fecha e rola até a seção.	RNF01	Aprovado
Destaque da seção	Destaca a seção visível e nenhuma seção no topo da página.	RNF01	Aprovado
Terminal de demonstração	Conclui a simulação e reinicia pelo botão “repetir”.	—	Aprovado
Indicadores	Contadores chegam aos valores finais.	—	Aprovado
Sem JavaScript	Todo o conteúdo permanece visível.	RNF07	Aprovado
Movimento reduzido	Animações desativadas e conteúdo visível.	RNF01	Aprovado

Fonte: Elaborado pelos autores (2026).

Os testes revelaram problemas, corrigidos e registrados no arquivo CHANGELOG.md (versão 0.1.1 do front-end), conforme o Quadro 24.

Quadro 24 — Problemas encontrados nos testes e correções

Problema encontrado	Correção aplicada
O texto da nota da abertura quebrava em colunas.	Texto agrupado em um único elemento, mantendo o ícone ao lado.
O menu marcava “FAQ” como ativo ao voltar ao topo da página.	O destaque passou a ser limpo quando a área inicial está visível.
O botão “Acompanhar o projeto” quebrava em duas linhas.	Botão com largura fixa, sem quebra de linha.
Com JavaScript desativado, os blocos animados ficavam invisíveis.	Animações condicionadas à classe js, adicionada apenas quando o script é executado.
Aberto diretamente de dentro do arquivo .zip, o index.html aparecia sem estilos.	Instrução de extração acrescentada ao README.md.

Fonte: Elaborado pelos autores (2026).

25.6 Evidências do funcionamento da interface

As Figuras 8 a 13 apresentam capturas de tela da página inicial em funcionamento, em computador, tablet e celular. Os arquivos originais estão na pasta docs/capturas/ do repositório.

Figura 8 — Página inicial: cabeçalho e abertura (computador, 1440 px)



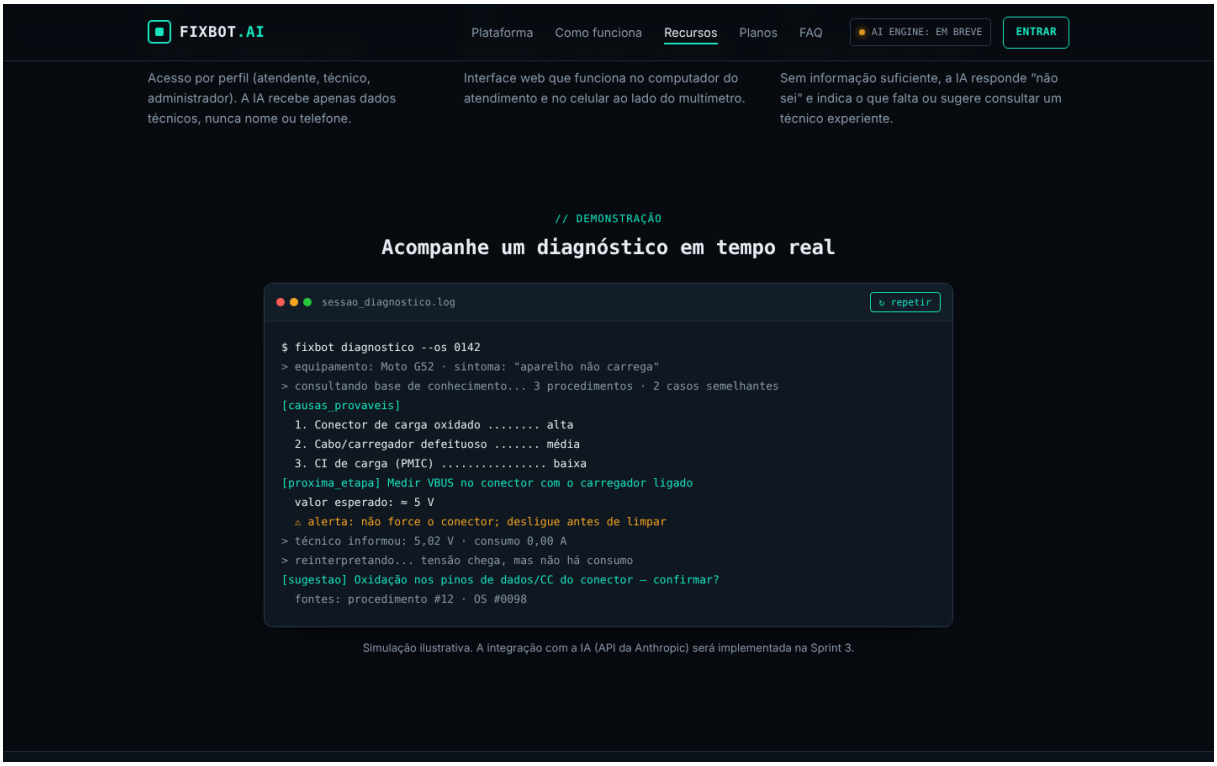
Fonte: Elaborado pelos autores (2026).

Figura 9 — Seções “Plataforma” e “Como funciona” (computador)



Fonte: Elaborado pelos autores (2026).

Figura 10 — Seção “Recursos” com o terminal de demonstração



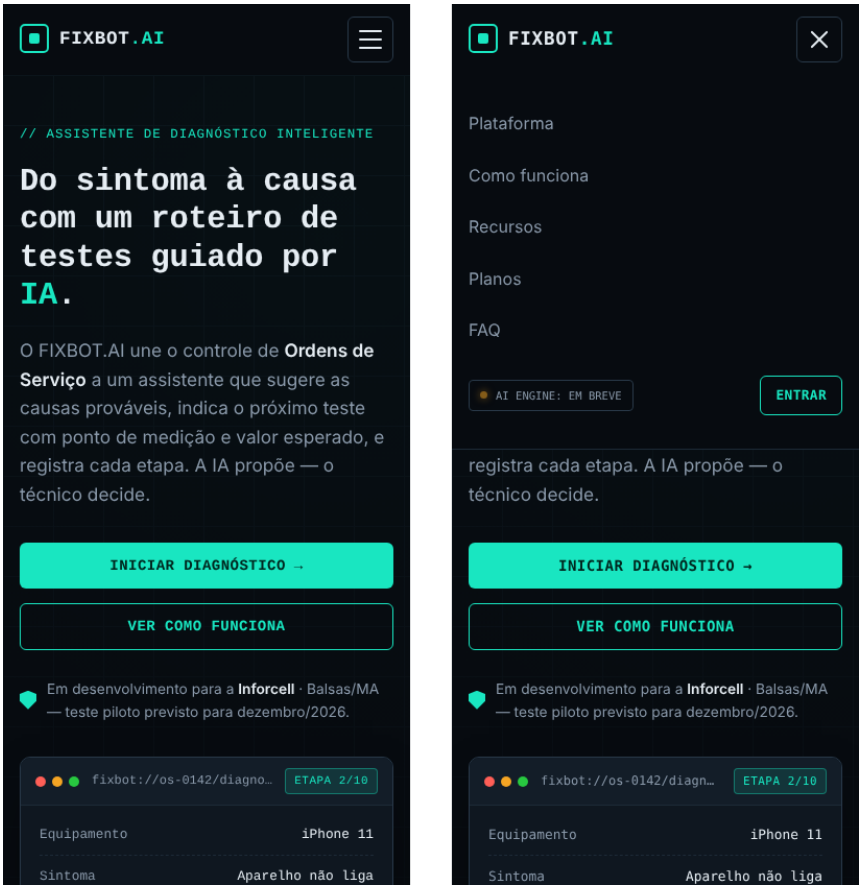
Fonte: Elaborado pelos autores (2026).

Figura 11 — Seções “Validação” e “Planos” (computador)



Fonte: Elaborado pelos autores (2026).

Figura 12 — Página inicial no celular (390 px): abertura e menu aberto



Fonte: Elaborado pelos autores (2026).

Figura 13 — Página inicial no tablet (820 px)



Fonte: Elaborado pelos autores (2026).

25.7 Execução e versão publicada

As instruções completas estão no arquivo frontend/README.md do repositório. Com o Node.js 18 ou superior instalado, executam-se na pasta frontend/ os comandos abaixo, e a aplicação fica disponível em http://localhost:5173.

```
npm install        # instala o TypeScript
npm run build      # compila src/main.ts para dist/main.js
npm run serve      # inicia o servidor local na porta 5173
```

25.8 Situação atualizada do cronograma

O Quadro 25 atualiza a situação das etapas do cronograma da Seção 21.

Quadro 25 — Cronograma atualizado (Versão 2.0)

Etapa	Atividade	Situação na Versão 2.0
1	Identificação do problema	Concluída
2	Levantamento com o cliente	Concluída
3	Definição da proposta	Concluída
4	Levantamento dos requisitos	Concluída
5	Prototipação	Concluída (protótipo de alta fidelidade no Figma)
6	Definição da arquitetura	Concluída (Quadro 20)
7	Definição do backend	Concluída (Node.js, Express e TypeScript)
8	Definição da API de IA	Concluída (API da Anthropic com RAG)
9	Desenvolvimento da interface	Em desenvolvimento (página inicial concluída)
10	Desenvolvimento do backend	Pendente
11	Implementação do banco de dados	Pendente
12	Implementação do gerenciamento de OS	Pendente
13	Implementação da IA	Pendente
14	Implementação do diagnóstico	Pendente
15	Testes	Em andamento (testes da interface, Seção 25.5)
16	Correções	Em andamento (Quadro 24)
17	Documentação final	Pendente
18	Apresentação	Pendente

Fonte: Elaborado pelos autores (2026).

26 CONSIDERAÇÕES SOBRE A VERSÃO 2.0

Na Versão 2.0, o projeto passou do planejamento à implementação com a entrega da página inicial da plataforma, desenvolvida em HTML, CSS e TypeScript conforme o protótipo e as tecnologias definidas na 1ª versão. A interface é responsiva, acessível por teclado, funciona sem JavaScript e foi verificada por testes automatizados e manuais, cujos problemas foram corrigidos e registrados.

Também foram registradas as definições que estavam pendentes na 1ª versão — o backend em Node.js com Express e TypeScript e a API de Inteligência Artificial da Anthropic — e atualizada a situação do cronograma. A estrutura de arquivos, os estilos e os componentes criados servirão de base para as telas de login, dashboard, Ordens de Serviço e diagnóstico, que serão implementadas nas próximas etapas, junto com a integração ao backend e à IA.

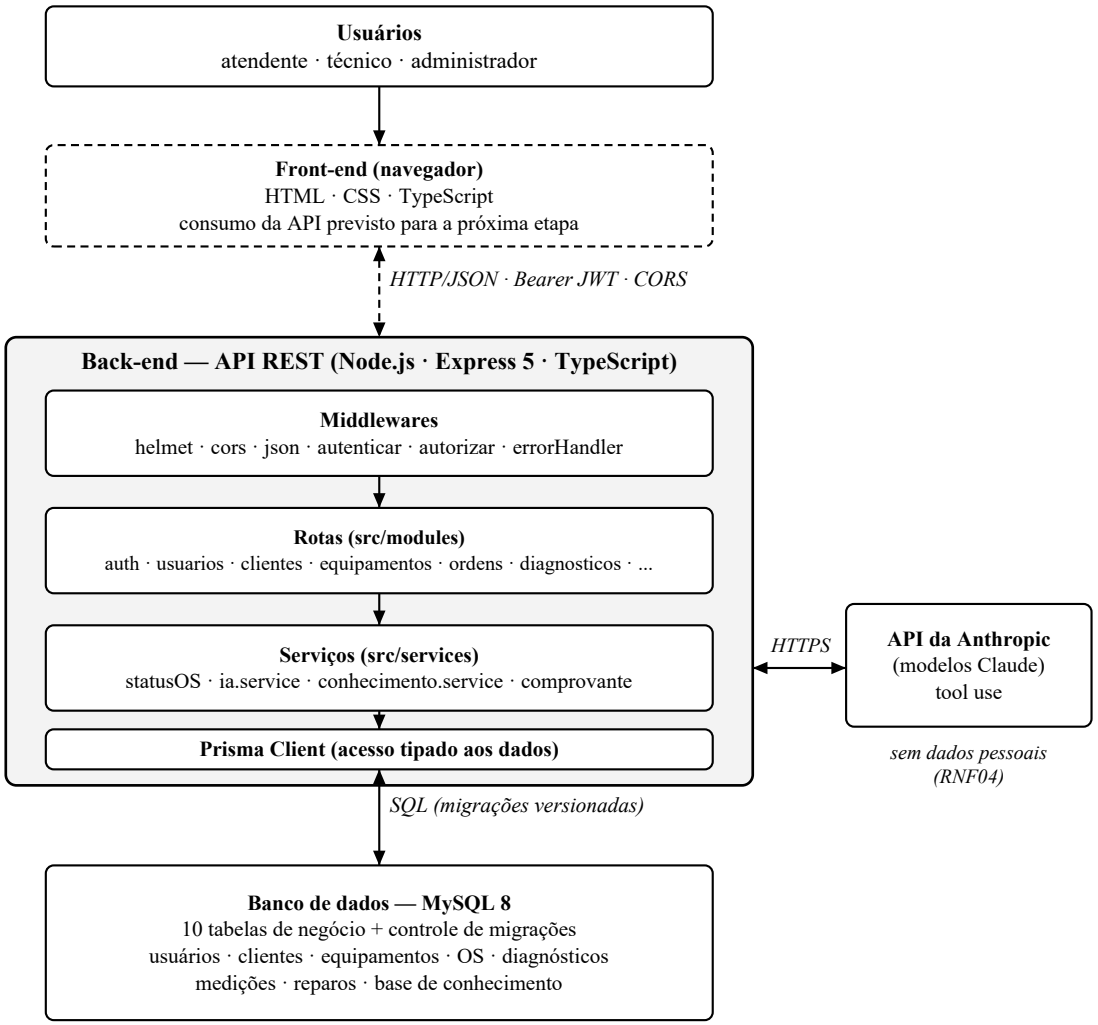
27 ARQUITETURA IMPLEMENTADA

Esta seção e as seguintes foram acrescentadas na Versão 3.0 e documentam a implementação da camada de back-end da plataforma: regras de negócio, persistência de dados, autenticação, autorização, validações, tratamento de erros e integração com o serviço de IA. O código está na pasta `backend/` do repositório informado na Seção 1.

27.1 Visão geral

A implementação manteve a arquitetura cliente-servidor em camadas da Seção 15. A API REST concentra as regras de negócio e o controle de acesso, persiste os dados em MySQL por meio do Prisma ORM e é o único componente que se comunica com o serviço de IA. A Figura 14 apresenta o que foi implementado. As linhas tracejadas indicam a ligação com o front-end, cujas telas passarão a consumir a API na próxima etapa.

Figura 14 — Arquitetura implementada



Fonte: Elaborado pelos autores (2026).

27.2 Organização do código

O código segue uma organização em camadas, conforme o RNF09. Cada requisição percorre os middlewares, chega à rota do recurso e, quando a regra é mais complexa, é delegada a um serviço. O acesso aos dados ocorre pelo Prisma Client (PRISMA DATA, 2026). O Quadro 26 descreve a estrutura.

Quadro 26 — Estrutura do back-end

Caminho	Responsabilidade
`prisma/schema.prisma`	Modelo de dados (entidades, relacionamentos, enumerações e índices).
`prisma/migrations/`	Script SQL versionado que cria o banco de dados.
`prisma/seed.ts`	Dados iniciais: usuários de teste de cada perfil, exemplos e artigos da base de conhecimento.
`src/app.ts, src/server.ts`	Montagem do Express, segurança, rotas e inicialização do servidor.
`src/config/env.ts`	Validação das variáveis de ambiente na inicialização.
`src/middlewares/`	Autenticação JWT, controle de perfil e tratamento centralizado de erros.
`src/modules/`	Rotas por recurso, com validação de entrada e regras de cada operação.
`src/services/`	Fluxo de status, integração com a IA, busca na base de conhecimento e comprovante.
`tests/`	Testes unitários e teste de ponta a ponta da API.
`requests.http, README.md`	Requisições de exemplo e documentação de instalação, execução e endpoints.

Fonte: Elaborado pelos autores (2026).

27.3 Tecnologias efetivamente utilizadas

As tecnologias previstas no Quadro 14 foram mantidas. O Quadro 27 apresenta as bibliotecas usadas e suas versões, conforme o arquivo `package-lock.json`.

Quadro 27 — Tecnologias utilizadas no back-end

Tecnologia	Versão	Uso no projeto
Node.js	20 ou superior	Ambiente de execução (testado na versão 24.21.0).
TypeScript	7.0.2	Linguagem do back-end, com tipagem estática.
Express	5.3.0	Servidor HTTP e rotas da API (OPENJS FOUNDATION, 2026).
Prisma ORM	6.19.3	Acesso tipado aos dados e migrações versionadas.
MySQL	8 (testado na 8.4.11)	Banco de dados relacional.
jsonwebtoken	9.0.3	Emissão e verificação dos tokens JWT.
bcryptjs	3.0.3	Hash de senhas.
Zod	3.25.76	Validação das entradas da API e das respostas da IA.
Helmet e cors	8.3.0 e 2.8.6	Cabeçalhos de segurança e controle das origens autorizadas.
SDK da Anthropic	1.4.0	Comunicação com a API de IA (ANTHROPIC, 2026).
Vitest e Supertest	5.1.0 e 7.3.1	Testes unitários e de integração da API (VITEST, 2026).
cross-env	10.1.0	Definição de variáveis nos scripts de teste em qualquer sistema operacional.

Fonte: Elaborado pelos autores (2026).

28 ALTERAÇÕES EM RELAÇÃO AO PLANEJAMENTO

As tecnologias definidas no planejamento não mudaram. As alterações se concentram em decisões de implementação e no escopo desta etapa, conforme o Quadro 28.

Quadro 28 — Alterações em relação ao planejamento inicial

Planejado	Implementado	Justificativa
Camadas de rotas, controladores e serviços (RNF09).	Rotas (com a lógica de controle), serviços e acesso a dados.	A API tem porte reduzido. Uma camada separada de controladores só repassaria chamadas; as regras mais complexas ficam nos serviços.
Recuperação de procedimentos da base por RAG (Quadro 20).	Busca por palavras-chave, com pontuação por sintoma, marca e modelo.	É suficiente para o volume atual da base e dispensa infraestrutura de busca vetorial. Pode evoluir sem alterar as rotas.
Técnico vê apenas as OS atribuídas a ele (HU04).	Técnico vê as OS atribuídas a ele e a fila de OS sem técnico.	Permite que o técnico assuma uma OS ainda não atribuída ao iniciar o diagnóstico. Ele continua sem acesso às OS de outros técnicos.
Fluxo de status da Seção 14.	Mantido, com o retorno de EM TESTES para EM REPARO, o cancelamento nas etapas iniciais e a passagem de CANCELADO para ENTREGUE.	Cobre a reprovação no teste e registra a devolução do aparelho. A Seção 14 previa ajustes conforme a necessidade real.
Reparo registrado leva a OS para EM TESTES (HU07).	O reparo é registrado e a mudança de status é uma ação separada, que exige reparo registrado.	Permite registrar mais de um reparo e mantém toda mudança de status explícita no histórico da OS.
Painel com filtro por técnico e período (HU08).	O painel mostra os totais; os filtros por técnico e período estão na listagem de OS.	Os dados já podem ser filtrados pela API. A combinação com o painel será definida na construção da tela.
Tela com opção “Esqueci minha senha” (Figura 4).	Troca de senha pelo próprio usuário autenticado e redefinição pelo administrador.	A recuperação por e-mail exige um serviço de envio, ainda não contratado.
Análise de imagens (Quadro 14) e integração USB (RF22 e RF23).	Não implementadas nesta etapa.	São funcionalidades complementares ou desejáveis, e não fazem parte do núcleo desta entrega.
Front-end consumindo a API (Seção 15).	API pronta e documentada; o front-end ainda contém apenas a página inicial.	Esta etapa tem como foco o back-end. As telas serão integradas na próxima etapa.

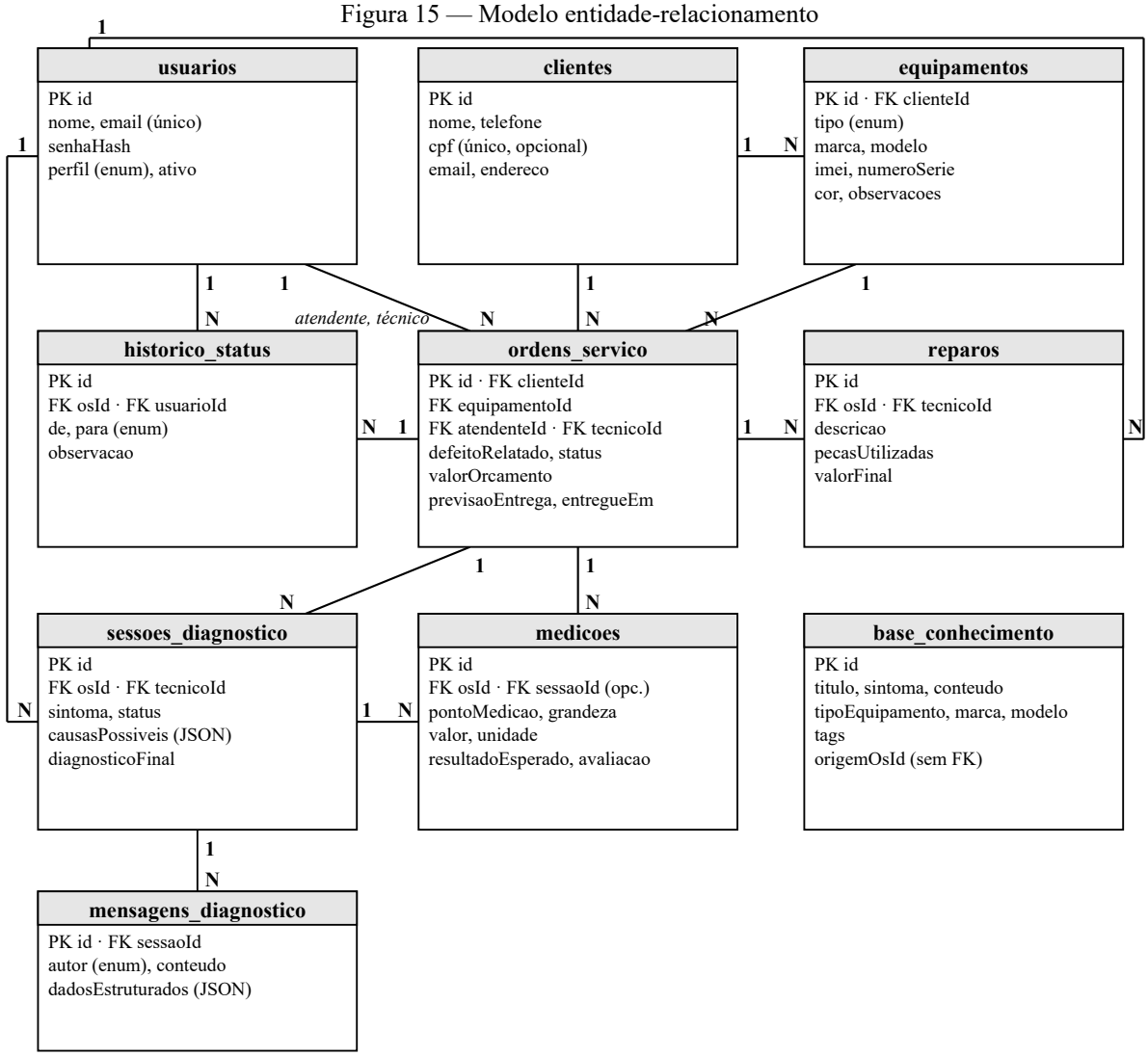
Fonte: Elaborado pelos autores (2026).

29 MODELAGEM DO BANCO DE DADOS

29.1 Modelo entidade-relacionamento

O banco foi modelado com 10 tabelas de negócio e 14 chaves estrangeiras. A Figura 15 apresenta as tabelas, os principais campos e os relacionamentos. A Ordem de Serviço é a entidade central: liga o cliente, o equipamento, o atendente e o técnico às etapas do atendimento.

Figura 15 — Modelo entidade-relacionamento



Fonte: Elaborado pelos autores (2026).

29.2 Dicionário das tabelas

Quadro 29 — Tabelas do banco de dados

Tabela	Conteúdo	Requisitos
usuarios	Contas de acesso, com perfil, hash da senha e indicação de conta ativa.	RF01, RF02
clientes	Dados de contato dos clientes. O CPF é opcional e único.	RF03
equipamentos	Aparelhos vinculados a um cliente, com índices para IMEI e número de série.	RF04, RF19
ordens_servico	Defeito relatado, status, técnico, orçamento, previsão e datas de conclusão.	RF05 a RF08
historico_status	Cada mudança de status, com autor, origem, destino e observação.	RF06
sessoes_diagnostico	Sessões de diagnóstico guiado por OS, com causas e diagnóstico final.	RF09 a RF13, RF17
mensagens_diagnostico	Conversa entre técnico e IA, incluindo a resposta estruturada em JSON.	RF15 a RF17
medicoes	Ponto de medição, grandeza, valor, unidade, resultado esperado e avaliação.	RF14
reparos	Reparo realizado, peças utilizadas e valor final.	RF18
base_conhecimento	Artigos técnicos e casos resolvidos usados como contexto para a IA.	Seção 17

Fonte: Elaborado pelos autores (2026).

29.3 Scripts de criação e configuração

O banco é criado pelo script SQL versionado em `backend/prisma/migrations/20261103143549_inicial/migration.sql`, aplicado com `npx prisma migrate deploy`. O comando `npm run db:seed` insere os dados iniciais. Na verificação, o Prisma não encontrou diferenças entre o banco criado pelo script e o modelo do arquivo `schema.prisma` (“No difference detected”).

30 PRINCIPAIS FUNCIONALIDADES IMPLEMENTADAS

O Quadro 30 relaciona as funcionalidades implementadas aos requisitos da Seção 10. A situação refere-se à API: as telas correspondentes ainda serão construídas.

Quadro 30 — Funcionalidades implementadas

Requisitos	Funcionalidade na API	Situação
RF01, RF02	Cadastro de usuários pelo administrador e três perfis de acesso.	Implementado
RF03, RF04	Cadastro de clientes e de equipamentos vinculados, com busca por nome, CPF ou telefone.	Implementado
RF05 a RF08	Abertura de OS com número sequencial, mudança de status, atribuição a técnico e consulta detalhada.	Implementado
RF09 a RF13	Diagnóstico guiado: causas possíveis ordenadas e um teste por vez, em formato estruturado.	Implementado; testado com IA simulada
RF14 a RF16	Registro de medições e resultados, com interpretação da IA e indicação do próximo teste.	Implementado; testado com IA simulada
RF17	Histórico das sessões e mensagens de diagnóstico.	Implementado
RF18, RF19	Registro de reparos e histórico completo do equipamento.	Implementado
RF20	Comprovante da OS em HTML, pronto para impressão.	Implementado
RF21	Busca de OS por número, cliente, telefone, IMEI, número de série ou modelo.	Implementado
Seção 17	Base de conhecimento consultada pela IA e alimentada com diagnósticos concluídos.	Implementado
HU08	Painel com OS por status, em aberto, atrasadas e por técnico.	Implementado (ver Quadro 28)
RF22, RF23	Integração com equipamentos e comunicação por USB.	Não implementado (desejável)

Fonte: Elaborado pelos autores (2026).

As principais regras de negócio aplicadas pela API estão no Quadro 31. Quando uma regra é violada, a operação é recusada e o banco permanece inalterado.

Quadro 31 — Regras de negócio

Regra	Resposta
Só são aceitas as transições de status previstas no fluxo. A resposta informa os status permitidos.	409
Cada status de destino só pode ser definido pelos perfis autorizados. A entrega, por exemplo, não pode ser registrada pelo técnico.	403
O diagnóstico só começa se a OS tiver técnico. Ao iniciar a sessão, o técnico assume a OS sem técnico.	409
A OS só vai para aprovação com o valor do orçamento informado.	409
A OS só vai para testes com ao menos um reparo registrado, e o reparo exige a OS em EM REPARO ou EM TESTES.	409
Uma OS entregue não pode mais ser alterada.	409
O técnico não acessa OS atribuídas a outro técnico e só pode assumir uma OS para si mesmo.	403
CPF e e-mail de usuário não podem ser duplicados.	409

Fonte: Elaborado pelos autores (2026).

31 AUTENTICAÇÃO, AUTORIZAÇÃO E TRATAMENTO DE ERROS

31.1 Autenticação

O usuário se autentica com e-mail e senha em `POST /api/auth/login`. As senhas são armazenadas apenas como hash bcrypt, com fator de custo 10 (RNF02). Em caso de sucesso, a API emite um token JWT (JONES; BRADLEY; SAKIMURA, 2015) assinado com a chave `JWT_SECRET` e com validade de 8 horas (RNF03), que deve ser enviado no cabeçalho `Authorization: Bearer <token>`. Por isso, o servidor não guarda estado de sessão. A mensagem de erro é a mesma para e-mail inexistente e senha incorreta, o que não revela quais contas existem. Em produção, a aplicação não inicia com a chave padrão de desenvolvimento.

31.2 Autorização

O middleware `autenticar` protege todas as rotas, exceto o login e a verificação de saúde. O middleware `autorizar` restringe cada operação aos perfis permitidos, conforme o Quadro 32.

Quadro 32 — Permissões por perfil

Operação	ADMIN	ATENDENTE	TÉCNICO
Gerenciar usuários	Sim	Não	Não
Cadastrar clientes e equipamentos	Sim	Sim	Não
Abrir OS e gerar comprovante	Sim	Sim	Não
Consultar OS	Todas	Todas	Atribuídas e sem técnico
Atribuir técnico	Sim	Sim	Apenas a si mesmo
Usar o diagnóstico com IA	Sim	Não	Sim
Registrar medições e reparos	Sim	Não	Sim
Mover a OS nas etapas técnicas	Sim	Não	Sim
Registrar aprovação do cliente (EM REPARO)	Sim	Sim	Sim
Registrar entrega	Sim	Sim	Não
Criar e editar artigos da base	Sim	Não	Sim
Excluir artigos da base	Sim	Não	Não

Fonte: Elaborado pelos autores (2026).

31.3 Validações e tratamento de erros

As entradas são validadas com Zod antes de qualquer gravação, e todos os erros passam por um único middleware, que devolve mensagens padronizadas em JSON. Detalhes internos são registrados apenas no servidor, conforme o Quadro 33.

Quadro 33 — Códigos de resposta de erro

Código	Situação
400	Dados inválidos, JSON malformado ou referência a registro inexistente.
401	Token ausente, inválido ou expirado; credenciais incorretas.
403	Perfil sem permissão para a operação.
404	Rota ou registro não encontrado.
409	Registro duplicado ou violação de regra de negócio.
500	Erro interno, sem exposição de detalhes ao cliente.
502	Falha na consulta à IA ou resposta fora do formato esperado.
503	Serviço de IA não configurado (chave de acesso ausente).

Fonte: Elaborado pelos autores (2026).

32 ARQUITETURA DE INTEGRAÇÃO E ENDPOINTS

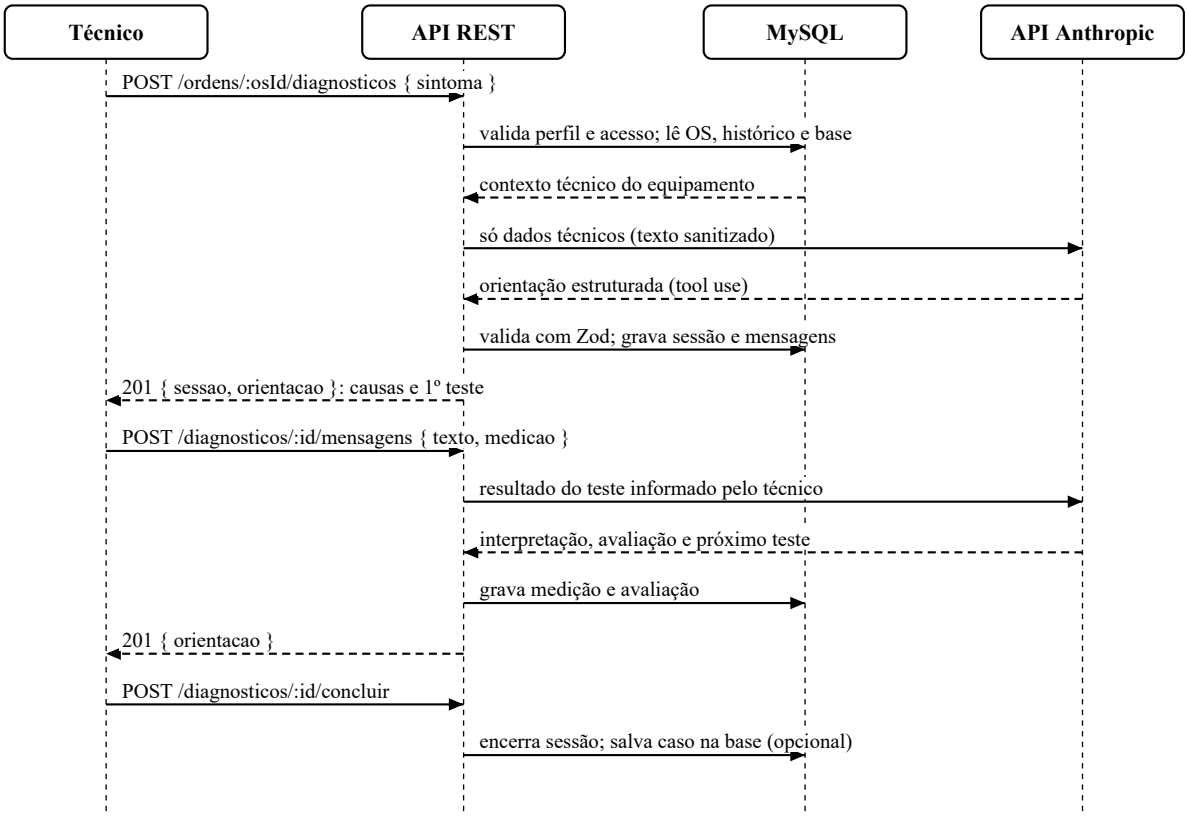
32.1 Integrações

A API se comunica por HTTP e JSON e integra três componentes: o cliente web, que usa as rotas REST e é autorizado pela lista de origens da variável `CORS_ORIGIN`; o banco MySQL, acessado pelo Prisma; e a API da Anthropic, único serviço externo consumido. A chave da IA fica apenas no servidor, e o front-end nunca a acessa.

32.2 Integração com o serviço de IA

A Figura 16 mostra a sequência de um diagnóstico guiado. A IA responde por meio de uma ferramenta com formato definido (tool use), e a resposta é validada com Zod antes de ser gravada. Assim, a API não depende de interpretar texto livre. O contexto enviado contém apenas tipo, marca e modelo do equipamento, sintoma, medições, diagnósticos anteriores do aparelho e artigos relevantes da base. O nome do cliente nunca é enviado, e CPF, telefone e e-mail digitados por engano são removidos antes do envio (RNF04).

Figura 16 — Sequência do diagnóstico guiado



Fonte: Elaborado pelos autores (2026).

O exemplo a seguir mostra o formato da orientação devolvida ao cliente da API:

```
{
  "mensagem": "Vamos começar pelo consumo na fonte de bancada.",
  "causasPossiveis": [
    { "causa": "Bateria defeituosa", "probabilidade": "alta", "justificativa": "..." },
    { "causa": "Curto na linha principal", "probabilidade": "media", "justificativa": "..." }
  ],
  "proximaEtapa": {
    "titulo": "Consumo na fonte",
    "procedimento": "Alimente a placa a 4,0 V e observe a corrente ao pressionar Power.",
    "instrumento": "Fonte de bancada",
    "pontoMedicao": "Conector da bateria",
    "resultadoEsperado": "Pico de 50–300 mA"
  },
  "interpretacaoUltimoResultado": null,
  "avaliacaoUltimaMedicao": null,
  "diagnosticoConcluido": false,
  "diagnosticoSugerido": null,
  "alertasSeguranca": []
}
```

32.3 Endpoints disponibilizados

O Quadro 34 lista os endpoints. Todos têm o prefixo `/api` e exigem token, exceto `/health` e `/auth/login`.

Quadro 34 — Endpoints da API

Método	Rota	Descrição	Perfis
GET	`/health`	Verificação de saúde da API.	Público
POST	`/auth/login`	Login; retorna o token e o usuário.	Público
GET	`/auth/me`	Dados do usuário logado.	Todos
PATCH	`/auth/me/senha`	Troca da própria senha.	Todos
GET	`/usuarios`	Lista de usuários.	ADMIN
POST	`/usuarios`	Cadastro de usuário.	ADMIN
PATCH	`/usuarios/:id`	Altera dados, perfil, senha ou situação.	ADMIN
GET	`/usuarios/tecnicos`	Técnicos ativos.	ADMIN, ATENDENTE
GET	`/clientes?busca=`	Busca por nome, CPF ou telefone.	Todos
GET	`/clientes/:id`	Cliente com equipamentos e OS.	Todos
POST, PATCH	`/clientes[/:id]`	Cadastro e alteração.	ADMIN, ATENDENTE
GET	`/equipamentos?clienteId=`	Lista de equipamentos.	Todos
GET	`/equipamentos/:id`	Dados do equipamento.	Todos
GET	`/equipamentos/:id/historico`	OS, diagnósticos, medições e reparos.	Todos
POST	`/equipamentos`	Cadastro de equipamento.	ADMIN, ATENDENTE
PATCH	`/equipamentos/:id`	Alteração de equipamento.	Todos
GET	`/ordens`	Lista com filtros: status, técnico, busca, período e página.	Todos*
GET	`/ordens/:id`	Detalhes e próximos status permitidos.	Todos*
POST	`/ordens`	Abertura de OS.	ADMIN, ATENDENTE
PATCH	`/ordens/:id`	Orçamento, previsão e observações.	Todos*
PATCH	`/ordens/:id/atribuir`	Atribuição de técnico.	Todos*
PATCH	`/ordens/:id/status`	Mudança de status.	Conforme o destino
POST	`/ordens/:id/medicoes`	Medição avulsa.	ADMIN, TECNICO
POST	`/ordens/:id/reparos`	Registro de reparo.	ADMIN, TECNICO
GET	`/ordens/:id/comprovante`	Comprovante em HTML.	ADMIN, ATENDENTE
POST	`/ordens/:osId/diagnosticos`	Inicia a sessão com a IA.	ADMIN, TECNICO

Método	Rota	Descrição	Perfis
GET	`/diagnosticos/:id`	Sessão, conversa e medições.	ADMIN, TECNICO
POST	`/diagnosticos/:id/mensagens`	Registra resultado e obtém o próximo teste.	ADMIN, TECNICO
POST	`/diagnosticos/:id/continuar`	Nova orientação após falha temporária da IA.	ADMIN, TECNICO
POST	`/diagnosticos/:id/concluir`	Encerra e, opcionalmente, salva na base.	ADMIN, TECNICO
GET	`/conhecimento`	Busca de artigos.	Todos
GET	`/conhecimento/:id`	Artigo.	Todos
POST, PATCH	`/conhecimento[/:id]`	Criação e edição de artigos.	ADMIN, TECNICO
DELETE	`/conhecimento/:id`	Exclusão de artigo.	ADMIN
GET	`/dashboard`	Indicadores do painel.	Todos

Fonte: Elaborado pelos autores (2026).

* O técnico só acessa OS atribuídas a ele ou sem técnico, e só pode atribuir uma OS a si mesmo.

32.4 Documentação técnica dos endpoints

A documentação dos endpoints está em `backend/README.md`, com os métodos, rotas, parâmetros, corpos das requisições, perfis de acesso e exemplos de resposta. O arquivo `backend/requests.http` traz requisições prontas para execução com a extensão REST Client do Visual Studio Code, percorrendo o fluxo de login, cadastro, abertura de OS e diagnóstico.

33 EVIDÊNCIAS DA INTEGRAÇÃO ENTRE OS COMPONENTES

As evidências foram coletadas em 03/11/2026, com a API em execução e conectada a um banco MySQL 8.4.11 criado pelos scripts do projeto. O Quadro 35 apresenta requisições HTTP reais e as respostas obtidas. Elas comprovam a integração entre API, autenticação, regras de negócio e banco de dados.

Quadro 35 — Requisições executadas na API em funcionamento

Requisição (prefixo /api)	Status	Resultado
`GET /health`	200	API no ar.
`GET /ordens (sem token)`	401	“Token de acesso não informado.”
`POST /auth/login (senha errada)`	401	“E-mail ou senha inválidos.”
`POST /auth/login (e-mail inválido)`	400	Erro de validação no campo email.
`POST /auth/login (atendente)`	200	Token JWT com validade de 28.800 s (8 horas).
`GET /usuarios (atendente)`	403	“Você não tem permissão para esta ação.”
`GET /usuarios (administrador)`	200	Lista dos usuários, sem o hash de senha.
`POST /clientes (técnico)`	403	Perfil sem permissão.
`POST /clientes (atendente)`	201	Cliente gravado, com CPF e telefone normalizados.
`POST /clientes (CPF repetido)`	409	“Registro duplicado.”, indicando a restrição do banco.
`POST /equipamentos`	201	Equipamento vinculado ao cliente.
`POST /ordens`	201	OS aberta com status RECEBIDO.
`PATCH /ordens/4/status (FINALIZADO)`	409	“Transição inválida: RECEBIDO → FINALIZADO.”, com os status permitidos.
`GET /ordens?busca=<IMEI>`	200	OS localizada pelo IMEI do equipamento.
`POST /ordens/4/diagnosticos (sem chave)`	503	“Serviço de IA não configurado.”
`PATCH /ordens/4/atribuir`	200	Técnico atribuído à OS.
`GET /ordens/4/comprovante`	200	Documento HTML do comprovante.
`GET /equipamentos/4/historico`	200	Histórico do equipamento com suas OS.
`GET /dashboard`	200	Contagem de OS por status e por técnico.
`GET /nao-existe`	404	“Rota não encontrada.”

Fonte: Elaborado pelos autores (2026).

A autorização de origens também foi verificada: uma requisição preliminar vinda de `http://localhost:5173` foi autorizada, enquanto outra de uma origem externa não recebeu a permissão de acesso. A Figura 17 mostra os dados gravados no banco após o teste de ponta a ponta. Ela registra as senhas em hash, todas as etapas do fluxo da OS, a medição avaliada e o caso resolvido salvo na base de conhecimento.

Figura 17 — Consultas ao banco de dados após a execução da API

```
mysql> SELECT id, nome, perfil, LEFT(senhaHash,7) hash_inicio, LENGTH(senhaHash) tam FROM usuarios;
| 1 | Administrador | ADMIN     | $2b$10$ | 60 |
| 2 | Atendente     | ATENDENTE | $2b$10$ | 60 |
| 3 | Técnico       | TECNICO   | $2b$10$ | 60 |

mysql> SELECT osId, de, para, observacao FROM historico_status WHERE osId = 3 ORDER BY id;
| 3 | NULL                | RECEBIDO             | Abertura da OS               |
| 3 | RECEBIDO            | EM_DIAGNOSTICO       | Diagnóstico iniciado         |
| 3 | EM_DIAGNOSTICO      | AGUARDANDO_APROVACAO | NULL                         |
| 3 | AGUARDANDO_APROVACAO | EM_REPARO           | Cliente aprovou por telefone |
| 3 | EM_REPARO           | EM_TESTES            | NULL                         |
| 3 | EM_TESTES           | FINALIZADO           | NULL                         |
| 3 | FINALIZADO          | ENTREGUE             | NULL                         |

mysql> SELECT pontoMedicao, valor, unidade, avaliacao FROM medicoes;
| Fonte de bancada | 1.2000 | A | ACIMA_DO_ESPERADO |

mysql> SELECT COUNT(*) FROM sessoes_diagnostico;    -- 1
mysql> SELECT COUNT(*) FROM mensagens_diagnostico;  -- 3

mysql> SELECT tags FROM base_conhecimento WHERE tags = 'caso-resolvido';
| caso-resolvido |
```

Fonte: Elaborado pelos autores (2026).

34 RESULTADOS DOS TESTES

34.1 Ambiente e abordagem

Os testes foram executados em 03/11/2026, em Windows 11, com Node.js 24.21.0 e MySQL 8.4.11, utilizando Vitest e Supertest. Para o banco foram aplicados os scripts de migração e o seed do próprio projeto. O serviço de IA foi substituído por um simulador nos testes automatizados, de modo que o resultado não dependa de um serviço externo e seja possível verificar o conteúdo enviado à IA.

34.2 Resultados

Figura 18 — Saída do comando npm run test:api

```
√ tests/unit.test.ts > fluxo de status da OS > segue a sequência do Documento Técnico
√ tests/unit.test.ts > fluxo de status da OS > bloqueia saltos e retrocessos indevidos
√ tests/unit.test.ts > fluxo de status da OS > respeita o perfil de quem move
√ tests/unit.test.ts > privacidade dos dados enviados à IA (RNF04) > remove CPF, e-mail e telefone
√ tests/unit.test.ts > privacidade dos dados enviados à IA (RNF04) > mantém valores de medição
√ tests/unit.test.ts > privacidade dos dados enviados à IA (RNF04) > alterna papéis e
    termina com o técnico
√ tests/unit.test.ts > busca na base de conhecimento > prioriza artigos do mesmo modelo e sintoma
√ tests/api.test.ts > API sem banco > health check responde
√ tests/api.test.ts > API sem banco > rotas protegidas exigem token (RNF02)
√ tests/api.test.ts > fluxo completo de uma OS com diagnóstico guiado >
    percorre atendimento → diagnóstico com IA → reparo → entrega

Test Files  2 passed (2)
     Tests  10 passed (10)
```

Fonte: Elaborado pelos autores (2026).

Quadro 36 — Resultados dos testes

Verificação	Critério	Resultado
Compilação	TypeScript e build sem erros.	Aprovado
Modelo de dados	Schema do Prisma válido e igual ao banco criado pelos scripts.	Aprovado
Migração e seed	Criação das tabelas e carga inicial sem erros.	Aprovado
Testes unitários	7 testes: fluxo de status, privacidade e busca na base.	Aprovado (7/7)
Testes da API sem banco	2 testes: saúde da API e exigência de token.	Aprovado (2/2)
Teste de ponta a ponta	OS completa, do atendimento à entrega, incluindo as recusas por perfil e por regra de negócio.	Aprovado (1/1)
Requisições manuais	20 requisições com os status esperados (Quadro 35).	Aprovado (20/20)

Fonte: Elaborado pelos autores (2026).

34.3 Situação dos requisitos não funcionais e limitações dos testes

Quadro 37 — Situação dos requisitos não funcionais

RNF	Situação na Versão 3.0	Observação
RNF01	Não avaliado	Depende das telas de abertura de OS.
RNF02	Atendido	Hash bcrypt confirmado no banco (Figura 17).
RNF03	Atendido	Perfis testados; token com validade de 8 horas.
RNF04	Atendido nos testes	Remoção por padrões de CPF, telefone e e-mail. Um nome digitado em texto livre não é detectado, e isso deve ser orientado aos técnicos.
RNF05	Parcial	Em ambiente local, nenhuma resposta da API passou de 86 ms. O tempo de carregamento das telas ainda não pode ser medido.
RNF06	Não medido	Exige consulta real à API da Anthropic.
RNF07	Não se aplica nesta etapa	Refere-se ao navegador; a página inicial foi avaliada na Versão 2.0.
RNF08	Pendente	A API responde por HTTP. O HTTPS será configurado na publicação, em servidor intermediário.
RNF09	Atendido	Código no GitHub, em camadas (ver Quadro 28).
RNF10	Pendente	O backup diário será definido com o ambiente de produção.

Fonte: Elaborado pelos autores (2026).

Os testes automatizados não comprovam a qualidade das orientações produzidas pelo modelo de IA, pois usam respostas simuladas. A consulta real depende de uma chave de acesso configurada em `ANTHROPIC_API_KEY`, e sem ela a API responde com o código 503, como demonstrado no Quadro 35. Os testes também não incluíram carga, usuários simultâneos nem ambiente de produção. Essas verificações ficam para a etapa de testes do cronograma.

35 PRINCIPAIS DIFICULDADES E SOLUÇÕES

Quadro 38 — Dificuldades encontradas e soluções adotadas

Dificuldade	Solução
Respostas da IA em texto livre são difíceis de gravar e de exibir de forma confiável.	Uso de tool use com formato definido, validação com Zod e resposta 502 em caso de formato inesperado. A rota `/continuar` permite pedir nova orientação após uma falha temporária.
Dados pessoais podem aparecer no texto digitado pelo técnico (RNF04).	Montagem do contexto apenas com dados técnicos e remoção de CPF, telefone e e-mail antes do envio, com testes que confirmam que os valores de medição são preservados.
Testar o diagnóstico sem depender de um serviço externo pago.	Função de consulta à IA substituível nos testes, permitindo simular as respostas e verificar o que seria enviado.
O script `npm run test:api` não funcionava no Windows, pela forma como definia a variável de ambiente.	Adoção do pacote cross-env, que funciona em qualquer sistema operacional.
Erro de caminho longo no Windows ao clonar o repositório em uma pasta muito profunda.	Uso de uma pasta com caminho mais curto.
Não havia MySQL instalado na máquina usada para validar a entrega.	Uso da versão portátil do MySQL 8.4, em porta local exclusiva, para executar migração, seed e testes.
A publicação automática do site falhava no repositório oficial, com o erro “Get Pages site failed”, por estar o GitHub Pages desativado.	Causa identificada. A ativação depende do responsável pelo repositório, em Settings → Pages → Source → GitHub Actions.
A revisão das Seções 1 a 24 alterou a numeração de requisitos, quadros e figuras em relação à Versão 2.0.	Ajuste das referências nas Seções 25 e 26, preservando o conteúdo e registrando a alteração no Histórico de Versões.

Fonte: Elaborado pelos autores (2026).

36 EXECUÇÃO E REPOSITÓRIO

36.1 Instalação e execução

As instruções completas estão em `backend/README.md`. Com o Node.js 20 ou superior e o MySQL 8 disponíveis, a API é executada com os comandos a seguir, na pasta `backend/`, e fica disponível em `http://localhost:3333/api`.

```
npm install
cp .env.example .env        # configure DATABASE_URL, JWT_SECRET e ANTHROPIC_API_KEY
npx prisma migrate deploy   # cria as tabelas
npm run db:seed             # insere os dados iniciais
npm run dev                 # inicia a API
npm test                    # testes sem banco
npm run test:api            # todos os testes, incluindo o fluxo com banco
```

36.2 Controle de versão

O back-end foi desenvolvido na branch `feature/backend-api`, conforme o fluxo da Seção 20, e submetido ao repositório oficial pelo pull request nº 2, a partir do fork de um dos integrantes. Na data deste documento, o pull request aguardava revisão, e a branch `develop` ainda precisava ser criada no repositório oficial. O Quadro 39 lista os commits, no padrão Conventional Commits.

Quadro 39 — Commits da entrega do back-end

Commit	Mensagem
`72db2a2`	chore(backend): configuração inicial do projeto
`2b07cd9`	feat(db): modelo de dados, migração inicial e seed
`c82fc40`	feat(auth): autenticação JWT, perfis de acesso e usuários
`ca85731`	feat(cadastros): clientes, equipamentos e histórico do aparelho
`b9f2a9f`	feat(os): ordens de serviço, fluxo de status e comprovante
`2003bb4`	feat(ia): diagnóstico guiado com a API da Anthropic e base de conhecimento
`b6647b1`	feat(api): painel e montagem da aplicação Express
`e0da7b9`	test(backend): testes unitários e fluxo completo da API
`2f5c6ba`	docs(backend): instruções de execução, endpoints e requisições de exemplo
`f43bee8`	docs: atualiza README com o back-end e a estrutura do repositório
`3b3ce12`	docs: atualiza a equipe do projeto no README

Fonte: Elaborado pelos autores (2026).

36.3 Entregáveis da Versão 3.0

Além do conteúdo da Seção 24, esta versão entrega: o código-fonte da API (`backend/`); os scripts do banco (`backend/prisma/`); as instruções de instalação e a documentação dos endpoints (`backend/README.md` e `backend/requests.http`); e este documento.

37 SITUAÇÃO ATUALIZADA DO CRONOGRAMA

O Quadro 40 atualiza a situação das etapas do cronograma da Seção 21 (Quadro 15).

Quadro 40 — Cronograma atualizado (Versão 3.0)

Etapa	Atividade	Situação na Versão 3.0
1 a 7	Problema, levantamento, proposta, requisitos, protótipo, arquitetura e API de IA	Concluídas
8	Desenvolvimento da interface	Em andamento (página inicial concluída)
9	Desenvolvimento do back-end e banco de dados	Concluída
10	Gerenciamento de OS	Concluída na API; telas pendentes
11	Integração com a IA e diagnóstico guiado	Implementada na API; falta validar com o modelo real e integrar às telas
12	Testes e correções	Em andamento (Seção 34)
13	Documentação final	Em andamento (esta versão)
14	Apresentação ao cliente e à banca	Pendente

Fonte: Elaborado pelos autores (2026).

38 CONSIDERAÇÕES SOBRE A VERSÃO 3.0

Na Versão 3.0, o projeto passou a contar com o núcleo funcional da plataforma: uma API REST com regras de negócio, persistência em MySQL, autenticação, autorização, validações, tratamento de erros e diagnóstico guiado integrado ao serviço de IA. Os testes automatizados e as requisições executadas contra um banco real confirmaram o funcionamento do fluxo completo de uma Ordem de Serviço, do atendimento à entrega.

Os próximos passos são construir as telas que consomem a API, validar o diagnóstico com o modelo real de IA e com técnicos da Inforcell, medir os tempos de resposta previstos nos RNF05 e RNF06 e preparar o ambiente de produção, com HTTPS e backup. A API já documentada oferece a base necessária para essas etapas.

REFERÊNCIAS

ANTHROPIC. **Claude API documentation**. Disponível em: https://docs.claude.com. Acesso em: 3 nov. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2024.

BEM. **BEM**: Block Element Modifier. Disponível em: https://getbem.com/. Acesso em: 5 out. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). **Diário Oficial da União**: Brasília, DF, 15 ago. 2018.

COHN, Mike. **User stories applied**: for agile software development. Boston: Addison-Wesley, 2004.

COOPER, Alan. **The inmates are running the asylum**: why high-tech products drive us crazy and how to restore the sanity. Indianapolis: Sams, 1999.

FIGMA, Inc. **Figma**: ferramenta colaborativa de design de interfaces. Disponível em: https://www.figma.com. Acesso em: 31 ago. 2026.

GITHUB, Inc. **GitHub Docs**. Disponível em: https://docs.github.com. Acesso em: 31 ago. 2026.

JONES, Michael B.; BRADLEY, John; SAKIMURA, Nat. **RFC 7519**: JSON Web Token (JWT). Internet Engineering Task Force, 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7519. Acesso em: 3 nov. 2026.

MDN WEB DOCS. **Documentação de HTML, CSS e JavaScript**. Mozilla Foundation. Disponível em: https://developer.mozilla.org. Acesso em: 31 ago. 2026.

MICROSOFT. **TypeScript documentation**. Disponível em: https://www.typescriptlang.org/docs/. Acesso em: 31 ago. 2026.

OPENJS FOUNDATION. **Express**: fast, unopinionated, minimalist web framework for Node.js. Disponível em: https://expressjs.com. Acesso em: 3 nov. 2026.

ORACLE CORPORATION. **MySQL 8.0 Reference Manual**. Disponível em: https://dev.mysql.com/doc/. Acesso em: 31 ago. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

PRISMA DATA. **Prisma ORM documentation**. Disponível em: https://www.prisma.io/docs. Acesso em: 3 nov. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

VITEST. **Vitest documentation**. Disponível em: https://vitest.dev. Acesso em: 3 nov. 2026.

W3C – WORLD WIDE WEB CONSORTIUM. **Web Content Accessibility Guidelines (WCAG) 2.2**. 2023. Disponível em: https://www.w3.org/TR/WCAG22/. Acesso em: 5 out. 2026.